SC520 视频通路交付包 · 评审卷宗

v1.1

HX1P35A 摄像头→HDMI 直通 + 饱和度/AWB 改动 —— 供外部独立评审

FPGA 创新赛视觉组（D:\fpga-ptz-tracker）

2026-10-05

目 录

一、评审范围与阅读指引 3
二、状态总览（截至 2026-10-05） 4
三、交付包结构与文件树 6
四、信号链架构与改动位置 7
五、自研与改动代码全文 8
六、验证证据 44
七、已知问题与开放问题（请评审重点） 46
八、复现速查 48
九、给评审者的提示词（v1.1 复核版，可直接复制发给 AI） 49

一、评审范围与阅读指引

本卷宗覆盖什么：交付包 `D:\SC520_display_burnpack` 的全部工程技术内容，以及仓库中与之对应的改动源码与验证证据。目标是让评审者不接触硬件、不打开 **TD** 也能完成技术评审。

文件归属（评审火力请集中在"我方"部分）：

归属	文件
我方新增	`axis_saturation_rgb96.v`、`tb_awb.v`、`tb_repack.v`、`tools/sat_tool.py`、三个 `burn_*.bat`、`px_serializer_fifo.v`、批次文档
我方修改	`awb.v`（#1 帧同步化；第2批：采样窗口/近黑剔除/frame_bad 门）、`isp_top.v`（饱和度实例；第2批：repack tlast 接入 + l_frame_bad）、`data_96bit_to_128bit`（第**2**批：间隙保相位 **+ tlast** 重建）、`design_top_wrapper.v`（第**2**批：坏帧标志接线，三行小改）、`tb_video_passthrough.v`（SAT_EN 开关）、`filelist_sim.f`
厂商原版（未动）	`user_source` 其余 96 个 RTL、`ip_source`、约束、`lab_ex1` 工程结构、`_hx1p35_docs` 官方文档、原版/彩条 bit

> **v1.1**（**2026-10-05**）变更说明：依首轮评审意见修订——(1) 更正两处过强表述：AWB"防异常帧污染"改为"防短/空/近白不足帧，坏帧由 frame_bad 门拦截（第2批）"；"tlast 已对齐"改为"第2批在 repack 内重建（与 O_tdata/O_tvalid 同拍）"。(2) 归入第2批的新增代码/测试/证据（5.9/5.10、6.6、7.1 更新、R1 重写）。(3) 改用 (1)(2)(3) 编号并补当前定版 bit（awbfs_b2）。

建议评审优先级：(1) AWB 帧同步化与第2批边界防护（窗口/暗剔除/坏帧门）的正确性； (2) repack 重构（气泡保相位 + tlast 重建）与 post-repack 记分板；(3) 流接口语义（tlast/repack）； (4) 第七章的 Q1~Q8 逐条结论（v1.1 已更新处置状态）。

二、状态总览（截至 2026-10-05）

2.1 比特流三代（板上/包内现状）

版本	内容	MD5	状态
camera_to_dsi_display.bit	厂商原版直通流（=best_result 同源）	2ef0ec47…	基线留档
…_sat_k21.bit	饱和度 1.31x（过渡版）	d211a715…	留档
…_sat_k28.bit	饱和度 1.75x（曾定版，已上板验收）	5f2a33c4…	留档
…_sat_k28_awbfs.bit	饱和度 1.75x + AWB 帧同步化（第1批）	38ded41ce7c3df94709ed15bf79d55f3	留档（未上板）
…_sat_k28_awbfs_b2.bit	上述 + 接口修复（第**2**批：**repack tlast/**异常帧门）	f1e3b561aca28d703f183935448f92d1	当前定版：已烧录实测（PRG-2014+返回 0，2026-10-05；目视验收进行中）

2.2 时序（无头两段流程：**syn_1 → phy_1**，厂商自带 **settings.cfg**）

构建	阶段	Setup WNS / TNS / 违规端点	Hold WNS
K21（1.31x）	phy final	+255ps / 0 / 0	+20ps / 0
K28（1.75x）	phy final	+703ps / 0 / 0	+20ps / 0
K28+AWB（第1批）	phy final	+176ps / 0 / 0	+20ps / 0
K28+AWB+接口（第2批，当前）	phy final	+143ps / 0 / 0	+20ps / 0

2.3 一致性（"工程源码 = 能重建板上定版"的核验）

位置	isp_top.v	awb.v	状态
仓库 01_video_passthrough/lab_ex1	✓ 含饱和度+K28	✓ 帧同步化	三处同哈希
仓库 03_td_rebuild_sat/lab_ex1_build（构建工作树）	✓	✓	（本表三次构建的来源）
交付包 02_TD_project_rebuild/lab_ex1	✓	✓	与上两处 MD5 一致

厂商原版基线唯一保留于仓库 _sc520_src/。

2.4 验证矩阵

- 单元 **TB**（tb_awb，28 项行为检查）→ 工程版 awb.v：全绿（6.1 全录）；
- 整链仿真（虚拟 RAW10，3 帧）→ ISP→饱和度→转储：PASS（几何与基线一致）；
- 位精确对拍（Python 同式参考）→ 饱和度级经整链：K=28 / K=50 全 PASS；
- 时序 → 三代构建全部 0 违规端点（2.2 表）；

- 烧录实测 → `PRG-2014 Chip validation success` + 100%（11~12s）。

三、交付包结构与文件树

```
SC520_display_burnpack\
├── 00_START_HERE_burn_guide.md      # 先读：包内容/接线/三种烧录方式/固化/故障速查
├── TD_burn_tutorial.pdf             # 烧录实战教程 14 页（试错+GUI逐步点击+CLI+固化）
├── video_passthrough_guide.pdf      # 详细版指导 9 页
├── burnpack_review.pdf              # 本卷宗
├── 01_bitstream_flash_direct\       # 成品 bit（原版/k21/k28/awbfs）+ QSPI 镜像 + 三个一键 bat
├── 02_TD_project_rebuild\lab_ex1\  # 完整 TD 工程（已含全部我方补丁，可重建定版）
│   ├── td_project\                 #   .al(已修XML) + Runs（厂商 best_result 留档）
│   └── user_source\                #   hdl_source 96 RTL + ip_source + 约束
├── 03_simulation_optional\sim\     # 整链仿真快照（TB 与仓库同步，含 SAT_EN 开关）
└── _hx1p35_docs\                    # 官方烧写/驱动文档留档
```

四、信号链架构与改动位置

```
SC520CS —MIPI RAW10—> dphy_rx/csi_unpacket —> demosaic —> AWB —> [我方] 饱和度级
                                                                  | axis_saturation_rgb96 (K=28)
                                                                  | data_96bit_to_128bit (96->128)
                                                                  |
        video_in ——DDR 帧缓存—— video_out —> hdmi_mixer —> hdmi_tx —> HDMI 720p60

改动点：awb.v（帧同步化）；isp_top.v（u_saturation 实例 + O_tlast = sat_O_tlast）
```

改动点	文件:位置	内容
(1) 饱和度级	`isp_top.v` u_saturation 实例	AWB 之后、重排之前；4 像素/拍、2 拍流水、无背压
(2) AWB 帧同步化	`awb.v` 三处（有"评审#1"注释）	上一帧统计锁存 / 商帧内恒定 / 帧边界 ±0.125x 步进
(3) tlast 对齐	`isp_top.v` 输出段	`O_tlast = sat_O_tlast`（原直取 awb，错位 3 拍）

五、自研与改动代码全文

5.1 axis_saturation_rgb96.v —— 饱和度增强级（新增，含评审#7 位宽修复）

我方新增。插在 ISP 链 AWB 之后、128bit 重排之前；Q4.4 增益 K（板上定版 28≈1.75x）；中间量 v 已扩为 12bit（评审 #7：原 10bit 在 K≥33 时先于限幅溢出）。该模块同时以独立文件（本节）与内嵌形式（5.3 isp_top.v 文件末尾）存在，两处文本一致。

（117 行，MD5 adc64c10）

```
// ============================================================================
// axis_saturation_rgb96.v — RGB 饱和度增强级（4 像素/拍，96-bit 流）
// ----------------------------------------------------------------------------
// 背景：板上画面"发灰、饱和度低"的根因是 ISP 链路只有 demosaic + AWB，
//       没有色彩校正（CCM）也没有饱和度处理；本模块补一个最小可用的饱和度级。
//
// 公式（全整数，与 tools/sat_tool.py 严格一致，可位精确对拍）：
//     Y   = (77*R + 150*G + 29*B) >> 8      // BT.601 亮度近似，8bit
//     d   = ch - Y                          // 有符号
//     out = clamp(Y + ((K_Q4 * d + 8) >> 4))// Q4.4 增益，16=1.00x, 28≈1.75x, 32=2.00x
//
// 接口风格与 isp_top 内 axis_gamma_rgb96 一致：单时钟、无背压、2 拍流水；
// tvalid/tlast/tuser 同步延迟 2 拍，与数据严格对齐（帧边界不受影响）。
//
// 字节序约定：与 AWB 输出一致 — 每 24bit 一个像素 {R[23:16], G[15:8], B[7:0]}，
//             [23:0]=第 0 像素 … [95:72]=第 3 像素；模块逐通道原位变换，不改布局。
// ============================================================================
module axis_saturation_rgb96 #(
    parameter [5:0] K_Q4 = 6'd21          // Q4.4 增益: 16=1.00x(恒等), 21≈1.31x, 28≈1.75x, 32=2.00x, 上限
 63≈3.94x
)(
    input  wire        I_clk,
    input  wire        I_rst_n,
    input  wire        I_tlast,
    input  wire        I_tuser,
    input  wire [95:0] I_tdata,
    input  wire        I_tvalid,
    output reg         O_tlast,
    output reg         O_tuser,
    output reg  [95:0] O_tdata,
    output reg         O_tvalid
);

    // ---------------- 第一级：每像素亮度 Y + 原色寄存 ----------------
    reg [7:0] s1_c [0:11];                // 4 像素 x {R,G,B}：3*i+0=R, +1=G, +2=B
    reg [7:0] s1_y [0:3];
    reg       s1_valid, s1_last, s1_user;
    integer   i;

    always @(posedge I_clk or negedge I_rst_n) begin
        if (!I_rst_n) begin
            s1_valid <= 1'b0;
            s1_last  <= 1'b0;
            s1_user  <= 1'b0;
        end else begin
            s1_valid <= I_tvalid;
            s1_last  <= I_tlast;
            s1_user  <= I_tuser;
            for (i = 0; i < 4; i = i + 1) begin
```

```
                s1_c[3*i+0] <= I_tdata[24*i + 23 -: 8];   // R
                s1_c[3*i+1] <= I_tdata[24*i + 15 -: 8];   // G
                s1_c[3*i+2] <= I_tdata[24*i +  7 -: 8];   // B
                s1_y[i]     <= (77  * I_tdata[24*i + 23 -: 8]
                              + 150 * I_tdata[24*i + 15 -: 8]
                              + 29  * I_tdata[24*i +  7 -: 8]) >> 8;
            end
        end
    end

    // ---------------- 第二级：饱和增益 + 限幅 ----------------
    function [7:0] sat_ch;
        input [7:0] c;
        input [7:0] y;
        reg signed [8:0]  d;
        reg signed [15:0] t;
        reg signed [11:0] v;                    // 12bit：K≤63 满量程（评审#7：原10bit 在 K≥33 时先于限幅溢出
)
        begin
            d = $signed({1'b0, c}) - $signed({1'b0, y});
            t = d * $signed({1'b0, K_Q4});        // 9bit x 7bit -> 16bit(signed)，|d|·K≤255×63=16065
            t = t + 16'sd8;                        // 四舍五入(+0.5)
            v = $signed({1'b0, y}) + (t >>> 4);    // 算术右移 = floor，与 Python >> 一致
            if (v < 0)          sat_ch = 8'd0;
            else if (v > 16'sd255) sat_ch = 8'd255;
            else                sat_ch = v[7:0];
        end
    endfunction

    reg [7:0] s2_o [0:11];
    reg       s2_valid, s2_last, s2_user;
    integer   j;

    always @(posedge I_clk or negedge I_rst_n) begin
        if (!I_rst_n) begin
            s2_valid <= 1'b0;
            s2_last  <= 1'b0;
            s2_user  <= 1'b0;
        end else begin
            s2_valid <= s1_valid;
            s2_last  <= s1_last;
            s2_user  <= s1_user;
            for (j = 0; j < 4; j = j + 1) begin
                s2_o[3*j+0] <= sat_ch(s1_c[3*j+0], s1_y[j]);
                s2_o[3*j+1] <= sat_ch(s1_c[3*j+1], s1_y[j]);
                s2_o[3*j+2] <= sat_ch(s1_c[3*j+2], s1_y[j]);
            end
        end
    end

    // ---------------- 输出寄存（总延迟 2 拍） ----------------
    always @(posedge I_clk or negedge I_rst_n) begin
        if (!I_rst_n) begin
            O_tvalid <= 1'b0;
            O_tlast  <= 1'b0;
            O_tuser  <= 1'b0;
            O_tdata  <= 96'd0;
        end else begin
            O_tvalid <= s2_valid;
            O_tlast  <= s2_last;
            O_tuser  <= s2_user;
            O_tdata  <= { s2_o[ 9], s2_o[10], s2_o[11],   // 像素3 (95:72)
                          s2_o[ 6], s2_o[ 7], s2_o[ 8],   // 像素2 (71:48)
```

```
                        s2_o[ 3], s2_o[ 4], s2_o[ 5],   // 像素1 (47:24)
                        s2_o[ 0], s2_o[ 1], s2_o[ 2] }; // 像素0 (23:0)
        end
    end

endmodule
```

5.2 awb.v —— 白平衡（改动版全文，板上真值；含#1 帧同步化与第2批边界防护）

厂商原版 + 第1批（#1 帧同步化）：上一帧统计锁存 / 商帧内恒定 / 帧边界 ±0.125x 步进、限幅 0.5~3.0x；+ 第2批：采样数上下界（20480~300000，防短帧与漏 SOF 并帧）+ 近黑剔除（rgb_sum>45）+ frame_bad 门（坏帧不采信）。改动段有注释标记；边界 TB 38 项全绿（5.4）。

（447 行，MD5 `7dbfaf09`）

```
/////////////////////////////////////////////////////////////////
Company : MiLianKe Electronic Technology Co., Ltd.
WebSite:https://www.milianke.com
TechWeb:https://www.uisrc.com
tmall-shop:https://milianke.tmall.com
jd-shop:https://milianke.jd.com
taobao-shop: https://milianke.taobao.com
Description:
The reference demo provided by Milianke is only used for learning.
We cannot ensure that the demo itself is free of bugs, so users
should be responsible for the technical problems and consequences
caused by the use of their own products.
@Author      :   XiaoQingquan
@Time        :   2024/10
version:     :
@Description :   在原基础上删去了一些赘余的代码，并且优化了时序，加入BRAM IP核，
                 让从除法器出来的每一个数据都可以对上相应的像素数据进行增益，使
                 得它的适用行更加广泛了。
////////////////////////////////////////////////////////////////*/
module awb  #(
    parameter IMG_HEIGHT = 1080,
    parameter IMG_WIDTH  = 1920
)
(
    input                   I_clk  ,
    input                   I_rst_n,


    input                   I_tlast  ,
    input                   I_tuser  ,
    input                   I_frame_bad ,   // 评审#2（第2批）：上一帧坏帧标志（帧首有效）
    input [95:0]            I_tdata  ,
    input                   I_tvalid ,
    output                  I_tready ,

    output                  O_tlast  ,
    output                  O_tuser  ,
    output [95:0]           O_tdata  ,
    output                  O_tvalid ,
    input                   O_tready
);

/////////////////////////////////////////////////////////////////
                        RGB565 RGB888
////////////////////////////////////////////////////////////////*/
```

```
    // //RGB565??????888
    // wire [4:0]  rgb565_r   = I_st_data[11+:5];
    // wire [5:0]  rgb565_g   = I_st_data[5+:6];
    // wire [4:0]  rgb565_b   = I_st_data[0+:5];

    // wire [23:0] rgb888     = {{rgb565_r[4:0],rgb565_r[2:0]},{rgb565_g[5:0],rgb565_g[1:0]},{rgb565_b[4:
0],rgb565_b[2:0]}};

    // wire [7:0]  rgb888_r   = rgb888[16+:8];
    // wire [7:0]  rgb888_g   = rgb888[8+:8] ;
    // wire [7:0]  rgb888_b   = rgb888[0+:8] ;

  /****************************************************************
                          中间信号
****************************************************************/


  reg         I_tvalid_r0,I_tvalid_r1;
  reg  [95:0] I_tdata_d0,I_tdata_d1;


  //累加RGB通道的 ??
  reg  [31:0]         sum_r[0:3];
  reg  [31:0]         sum_g[0:3];
  reg  [31:0]         sum_b[0:3];


  wire  [95:0]        data_d0 ;
  wire                valid_d0;
  wire                I_tuser_r0;

  reg                 I_tuser_r1;

  reg   [95:0]        data_d1 ;
  reg                 valid_d1;
  reg   [9:0]         rgb_sum[0:3];//每个像素点RGB的和
  /****************************************************************
                           平均值计 ??
****************************************************************/

// 对输入数据和有效信号进行寄存，形成流水线
always @(posedge I_clk or negedge I_rst_n) begin
    if (!I_rst_n) begin
        I_tdata_d0  <= 0;
        I_tdata_d1  <= 0;
        {I_tvalid_r0,I_tvalid_r1} <= 0;
    end else begin
        I_tdata_d0  <= I_tdata   ;
        I_tdata_d1  <= I_tdata_d0;
        I_tvalid_r0 <= I_tvalid;
        I_tvalid_r1 <= I_tvalid_r0;
    end
end

// 将输入数据拆分为4个像素的RGB分量
wire [7:0] rgb888_r_d[0:3];
wire [7:0] rgb888_g_d[0:3];
wire [7:0] rgb888_b_d[0:3];

assign  rgb888_r_d[0] = I_tdata[16+:8];
assign  rgb888_g_d[0] = I_tdata[8+:8];
assign  rgb888_b_d[0] = I_tdata[0+:8];
```

```
// 将输入数据拆分为4个像素的RGB分量
wire [7:0] rgb888_r_d0[0:3];
wire [7:0] rgb888_g_d0[0:3];
wire [7:0] rgb888_b_d0[0:3];

assign  rgb888_r_d0[0] = I_tdata_d0[16+:8];
assign  rgb888_g_d0[0] = I_tdata_d0[8+:8];
assign  rgb888_b_d0[0] = I_tdata_d0[0+:8];


        ///削弱白色
        always @(posedge I_clk or negedge I_rst_n) begin
            if(!I_rst_n) begin
                rgb_sum[0] <= 0;
            end
            else if(I_tvalid)begin
                rgb_sum[0] <= rgb888_r_d[0] + rgb888_g_d[0] + rgb888_b_d[0];
            end
        end

        // ---- 采样窗口常量（第2批）：近黑/近白剔除 + 采样数上下界 ----
        localparam [31:0] STAT_MIN_BEATS = 32'd20480;   // 下界：防短帧/空帧/近白不足帧
        localparam [31:0] STAT_MAX_BEATS = 32'd300000;  // 上界：防漏 SOF 的跨帧并帧
                                                        // （720p 整帧 230400；两帧并帧 460800 > 上界 → 拒
)
        localparam [11:0] RGB_SUM_MIN    = 12'd45;      // 近黑剔除下限（评审#2：低照度噪声驱动比值病态）

        // 对每个通道的值进行累加，用于计算平均值
        always @(posedge I_clk or negedge I_rst_n) begin
            if (!I_rst_n || I_tuser) begin
                // 复位或行开始信号有效时，清零累加器
                sum_r[0] <= 1;
                sum_g[0] <= 1;
                sum_b[0] <= 1;
            end else if (I_tvalid_r0 && (rgb_sum[0] < 720) && (rgb_sum[0] > RGB_SUM_MIN)) begin
                // 当输入有效时，累加RGB分量的值
                sum_r[0] <= sum_r[0] + rgb888_r_d0[0];
                sum_g[0] <= sum_g[0] + rgb888_g_d0[0];
                sum_b[0] <= sum_b[0] + rgb888_b_d0[0];
            end
        end

        // ---- 评审#1（2026-10-04 第1批）：上一帧统计锁存 ----
        // 帧 N 使用帧 N-1 的最终统计（帧内恒定）；仅当本帧累计到足够多
        // 非白像素才采信，防异常/空帧污染（阈值按 720p 帧量级的 ~9%）。
        reg [31:0] sum_r_p, sum_g_p, sum_b_p;      // 上一帧最终统计（初值 1 → 商恰为 1.0x）
        reg [31:0] stat_beats;                     // 本帧已累计的合格样本拍数
        always @(posedge I_clk or negedge I_rst_n) begin
            if (!I_rst_n) begin
                sum_r_p    <= 32'd1;
                sum_g_p    <= 32'd1;
                sum_b_p    <= 32'd1;
                stat_beats <= 32'd0;
            end else begin
                if (I_tvalid_r0 && (rgb_sum[0] < 720) && (rgb_sum[0] > RGB_SUM_MIN))
                    stat_beats <= stat_beats + 32'd1;
                if (I_tuser) begin
                    if ((stat_beats > STAT_MIN_BEATS) && (stat_beats < STAT_MAX_BEATS) && !I_frame_bad) b
egin
                        // 与 sum_*[0] 复位同拍：非阻塞读旧值 = 上一帧最终值；
                        // 异形帧（过短/过长）与坏帧（MIPI 错误锁存）一律不采信
                        sum_r_p <= sum_r[0];
```

```
                        sum_g_p <= sum_g[0];
                        sum_b_p <= sum_b[0];
                    end
                    stat_beats <= 32'd0;
                end
            end
        end

  /*****************************************************************
                            计算增益
*****************************************************************/

  wire [47:0] dout_R[3:0];
  wire [47:0] dout_B[3:0];

  wire done[3:0];

  wire [47:0] in_r_x1;
  wire [47:0] in_b_x1;
  assign in_r_x1 = dout_R[0][47:0];
  assign in_b_x1 = dout_B[0][47:0];



/*****************************************************************
                    IP 调用
*****************************************************************/
    //为了保证合理的延迟，这里使用了除法器

      // 评审#1：输入改用"上一帧统计"（帧内恒定）—增益不再随帧内累加漂移
      divider div_gen_G_R_r1 (
          .clk(I_clk),  // input wire aclk
          .rst(!I_rst_n),
          .start(I_tvalid_r1),
          .denominator(sum_r_p),  // input wire [31 : 0] s_axis_divisor_tdata
          .numerator({sum_g_p,16'd0}),  // input wire [47 : 0] s_axis_dividend_tdata
          .quotient(dout_R[0]),  // output wire [79 : 0] m_axis_dout_tdata
          .done(done[0])
      );

      divider divide_G_B_r1 (
          .clk(I_clk),  // input wire aclk
          .rst(!I_rst_n),
          .start(I_tvalid_r1),
          .denominator(sum_b_p),  // input wire [31 : 0] s_axis_divisor_tdata
          .numerator({sum_g_p,16'd0}),  // input wire [47 : 0] s_axis_dividend_tdata
          .quotient(dout_B[0]),  // output wire [79 : 0] m_axis_dout_tdata
          .done()
      );


    signal_delay #(
        .IMG_HEIGHT   (IMG_HEIGHT),
        .IMG_WIDTH    (IMG_WIDTH ),
        .DATA_WIDTH   (96  ),
        .DELAY_CYCLE  (50)
    )signal_delay_d (
        /*input                       */ .I_clk  (I_clk  ),
        /*input                       */ .I_rst_n(I_rst_n),
        /*input                       */ .I_tuser(I_tuser),  //synthesis keep
        /*input  wire                 */ .I_valid(I_tvalid_r1),  //synthesis keep
        /*input  wire [DATA_WIDTH-1:0]*/ .I_data (I_tdata_d1 ),   //synthesis keep
        /*output wire                 */ .O_valid(valid_d0   ),  //synthesis keep
```

```
        /*output wire [DATA_WIDTH-1:0]*/ .O_data (data_d0    ),   //synthesis keep
        /*output wire                 */ .O_tuser(I_tuser_r0 )
    );



/****************************************************************

****************************************************************/

    // ---- 评审#1（2026-10-04 第1批）：整帧恒定增益 + 限幅 + 跨帧平滑 ----
    // 修复点：原代码在 I_tuser 处把增益复位为 1.0、帧内随统计持续更新 →
    //         每帧顶部"无色偏→修正色"的色彩瞬态。现改为：
    //   (1) 目标增益 = clamp(上一帧统计商, 0.5x~3.0x)（商基于 prev 统计，帧内恒定）；
    //   (2) 增益仅在帧边界（tuser）向目标按 ±GAIN_STEP 步进（跨帧平滑，无帧内变化）；
    //   (3) 复位只在 !rst_n 时（不再每帧复位）。
    localparam [19:0] GAIN_MIN  = 20'h08000;   // 0.5x
    localparam [19:0] GAIN_MAX  = 20'h30000;   // 3.0x
    localparam [19:0] GAIN_STEP = 20'h02000;   // 0.125x/帧

    wire [19:0] tgt_r = (in_r_x1 > {28'd0, GAIN_MAX}) ? GAIN_MAX :
                        (in_r_x1 < {28'd0, GAIN_MIN}) ? GAIN_MIN : in_r_x1[19:0];
    wire [19:0] tgt_b = (in_b_x1 > {28'd0, GAIN_MAX}) ? GAIN_MAX :
                        (in_b_x1 < {28'd0, GAIN_MIN}) ? GAIN_MIN : in_b_x1[19:0];

    reg  [19:0] tgt_r_r, tgt_b_r;
    always @(posedge I_clk or negedge I_rst_n)begin
        if(!I_rst_n)begin
            tgt_r_r <= 20'h10000;              // 起始 1.0x
            tgt_b_r <= 20'h10000;
        end
        else if(done[0])begin                  // 商每拍刷新（值帧内恒定），寄存一档
            tgt_r_r <= tgt_r;
            tgt_b_r <= tgt_b;
        end
    end

    reg  [19:0]   in_r_r1;
    reg  [19:0]   in_b_r1;
    always @(posedge I_clk or negedge I_rst_n)begin
        if(!I_rst_n)begin
            in_r_r1 <= 20'h10000;              // 仅复位为 1.0x（不再每帧复位）
            in_b_r1 <= 20'h10000;
        end
        else if(I_tuser)begin                  // 帧边界：向目标步进（跨帧平滑）
            if(tgt_r_r > in_r_r1 + GAIN_STEP)       in_r_r1 <= in_r_r1 + GAIN_STEP;
            else if(tgt_r_r + GAIN_STEP < in_r_r1)  in_r_r1 <= in_r_r1 - GAIN_STEP;
            else                                    in_r_r1 <= tgt_r_r;
            if(tgt_b_r > in_b_r1 + GAIN_STEP)       in_b_r1 <= in_b_r1 + GAIN_STEP;
            else if(tgt_b_r + GAIN_STEP < in_b_r1)  in_b_r1 <= in_b_r1 - GAIN_STEP;
            else                                    in_b_r1 <= tgt_b_r;
        end
    end


    ////信号同步
    always @(posedge I_clk or negedge I_rst_n) begin
      if (!I_rst_n) begin
          valid_d1 <= 0;
          data_d1  <= 0;
          I_tuser_r1 <= 0;
      end else begin
          valid_d1 <= valid_d0;
```

```
          data_d1  <= data_d0 ;
          I_tuser_r1 <= I_tuser_r0;
      end
    end
/*************************************************************

*************************************************************/

    reg [27:0] R_new_r1;
    reg [27:0] G_new_r1;
    reg [27:0] B_new_r1;

    reg [27:0] R_new_r2;
    reg [27:0] G_new_r2;
    reg [27:0] B_new_r2;

    reg [27:0] R_new_r3;
    reg [27:0] G_new_r3;
    reg [27:0] B_new_r3;

    reg [27:0] R_new_r4;
    reg [27:0] G_new_r4;
    reg [27:0] B_new_r4;


    wire   [7:0] rgb888_r_d1[0:3];
    wire   [7:0] rgb888_g_d1[0:3];
    wire   [7:0] rgb888_b_d1[0:3];

    assign  rgb888_r_d1[0] = data_d1[16+:8];
    assign  rgb888_g_d1[0] = data_d1[8+:8];
    assign  rgb888_b_d1[0] = data_d1[0+:8];

    assign  rgb888_r_d1[1] = data_d1[40+:8];
    assign  rgb888_g_d1[1] = data_d1[32+:8];
    assign  rgb888_b_d1[1] = data_d1[24+:8];

    assign  rgb888_r_d1[2] = data_d1[64+:8];
    assign  rgb888_g_d1[2] = data_d1[56+:8];
    assign  rgb888_b_d1[2] = data_d1[48+:8];

    assign  rgb888_r_d1[3] = data_d1[88+:8];
    assign  rgb888_g_d1[3] = data_d1[80+:8];
    assign  rgb888_b_d1[3] = data_d1[72+:8];
    always @(posedge I_clk or negedge I_rst_n)begin
        if(!I_rst_n)begin
            R_new_r1 <= 0;
            G_new_r1 <= 0;
            B_new_r1 <= 0;

            R_new_r2 <= 0;
            G_new_r2 <= 0;
            B_new_r2 <= 0;

            R_new_r3 <= 0;
            G_new_r3 <= 0;
            B_new_r3 <= 0;

            R_new_r4 <= 0;
            G_new_r4 <= 0;
            B_new_r4 <= 0;
        end
        else if(valid_d1) begin
```

```
            R_new_r1 <= in_r_r1 * rgb888_r_d1[0];
            G_new_r1 <=           rgb888_g_d1[0];
            B_new_r1 <= in_b_r1 * rgb888_b_d1[0];

            R_new_r2 <= in_r_r1 * rgb888_r_d1[1];
            G_new_r2 <=           rgb888_g_d1[1];
            B_new_r2 <= in_b_r1 * rgb888_b_d1[1];

            R_new_r3 <= in_r_r1 * rgb888_r_d1[2];
            G_new_r3 <=           rgb888_g_d1[2];
            B_new_r3 <= in_b_r1 * rgb888_b_d1[2];

            R_new_r4 <= in_r_r1 * rgb888_r_d1[3];
            G_new_r4 <=           rgb888_g_d1[3];
            B_new_r4 <= in_b_r1 * rgb888_b_d1[3];
        end
    end

/*****************************************************************
                                ???
*****************************************************************/

    wire [7:0] R_new_x1;
    wire [7:0] G_new_x1;
    wire [7:0] B_new_x1;

    wire [7:0] R_new_x2;
    wire [7:0] G_new_x2;
    wire [7:0] B_new_x2;

    wire [7:0] R_new_x3;
    wire [7:0] G_new_x3;
    wire [7:0] B_new_x3;

    wire [7:0] R_new_x4;
    wire [7:0] G_new_x4;
    wire [7:0] B_new_x4;

    assign R_new_x1 = (R_new_r1[27:24])?255:R_new_r1[23:16];
    assign G_new_x1 =  G_new_r1;
    assign B_new_x1 = (B_new_r1[27:24])?255:B_new_r1[23:16];

    assign R_new_x2 = (R_new_r2[27:24])?255:R_new_r2[23:16];
    assign G_new_x2 =  G_new_r2;
    assign B_new_x2 = (B_new_r2[27:24])?255:B_new_r2[23:16];

    assign R_new_x3 = (R_new_r3[27:24])?255:R_new_r3[23:16];
    assign G_new_x3 =  G_new_r3;
    assign B_new_x3 = (B_new_r3[27:24])?255:B_new_r3[23:16];

    assign R_new_x4 = (R_new_r4[27:24])?255:R_new_r4[23:16];
    assign G_new_x4 =  G_new_r4;
    assign B_new_x4 = (B_new_r4[27:24])?255:B_new_r4[23:16];
/*****************************************************************
                            ┌   ź
*****************************************************************/

    reg valid_d2,I_tuser_r2;
    always @(posedge I_clk or negedge I_rst_n) begin
        if(!I_rst_n)begin
            valid_d2 <= 0;
            I_tuser_r2 <= 0;
        end
```

```
        else begin
            valid_d2 <= valid_d1;
            I_tuser_r2 <= I_tuser_r1;
        end
    end


    assign  O_tlast  = (valid_d1 == 0) && (valid_d2 == 1);
    // video_in uses frame_start to reset its asynchronous input FIFO.
    // Keep this pulse ahead of the delayed AWB pixels so the reset has
    // completed before the first output data reaches the FIFO.
    assign  O_tuser  = I_tuser;
    assign  O_tvalid = valid_d2;

    assign  O_tdata = O_tvalid?{R_new_x4,G_new_x4,B_new_x4,
                        R_new_x3,G_new_x3,B_new_x3,
                        R_new_x2,G_new_x2,B_new_x2,
                        R_new_x1,G_new_x1,B_new_x1}:0;
    // assign O_t_data = {R_new[7-:5],G_new[7-:6],B_new[7-:5]};
    assign  I_tready = O_tready;

endmodule
```

5.3 isp_top.v —— ISP 顶层（饱和度实例 + 第2批接口接线；全文）

厂商原版 + 我方改动：u_saturation 实例（AWB 之后）；第2批：repack 的 I_96b_tlast/O_128b_tlast 接入、O_tlast = repack_O_tlast（与 O_tdata/O_tvalid 同拍）、新增 I_frame_bad 端口。

（406 行，MD5 `5680e330`）

```
module isp_top (
    input  axi4s_video_aclk,
    input  I_rst_n ,
    input  I_tlast ,
    input  I_tuser ,
    input [39:0]  I_tdata ,
    input  I_tvalid ,
    input [9:0]  I_tdest ,
    input  I_frame_bad ,   // 评审#2（第2批）：上一帧坏帧（帧首有效）
    input  O_tready ,
    output [127:0]  O_tdata ,
    output  O_tlast ,
    output  O_tuser ,
    output  O_tvalid ,
    output  I_tready
);

wire         m_aixs_tvalid;  //synthesis keep
  wire [127:0] m_aixs_tdata;  //synthesis keep
  wire         m_aixs_tuser;  //synthesis keep
  wire         m_aixs_tlast;  //synthesis keep
  wire         m_aixs_tready;  //synthesis keep

  wire [ 95:0] m_aixs_tdata_96;  //synthesis keep


  wire         awb_O_tlast;  //synthesis keep
  wire         awb_O_tuser;  //synthesis keep
  wire [ 95:0] awb_O_tdata;  //synthesis keep
  wire         awb_O_tvalid;  //synthesis keep
  wire         awb_O_tready;  //synthesis keep
```

```
  wire         awb_O_tuser_128;  //synthesis keep
  wire         awb_O_tvalid_128;  //synthesis keep
  wire [127:0] awb_O_tdata_128;  //synthesis keep
  wire         repack_O_tlast;      // 第2批：repack 重建的行尾

  assign awb_O_tready = O_tready;
  assign O_tdata  = awb_O_tdata_128;
  // 评审#5（第1批）+ 评审#4/#8（2026-10-05 第2批）：tlast 在 repack（96→128）内重建，
  // 与 O_tdata/O_tvalid 同拍输出（行尾落在含该行最后一拍的输出字上）。
  assign O_tlast  = repack_O_tlast ;
  assign O_tuser  = awb_O_tuser_128    ;
  assign O_tvalid = awb_O_tvalid_128   ;

  //assign m_aixs_tready = O_tready;
  //assign O_tdata  = m_aixs_tdata;
  //assign O_tlast  = m_aixs_tlast    ;
  //assign O_tuser  = m_aixs_tuser    ;
  //assign O_tvalid = m_aixs_tvalid   ;


demosaic #(
    .IMG_HEIGHT          (720 ),  // 图像高度
    .IMG_WIDTH           (1280),  // 图像宽度
    .data_complete_delay (50  ),
    .BAYER_MODE          ("BGGR")
)
u_demosaic
(
    .I_clk    (axi4s_video_aclk) ,   // 时钟信号
    .I_rst_n  (I_rst_n) ,   // 复位信号，低有效
    .axi4s_video_tdata  (I_tdata) ,  // AXI4-Stream视频数据
    .axi4s_video_tdest  (I_tdest) ,
    .axi4s_video_tlast  (I_tlast) ,  // 行结束信号
    .axi4s_video_tvalid (I_tvalid) ,  // 数据有效信号
    .axi4s_video_tuser  (I_tuser) ,  // 帧开始信号
    .axi4s_video_tready (I_tready) ,  // 从模块准备好接受数据
    .O_tlast   (m_aixs_tlast) ,  // 输出行结束信号
    .O_tuser   (m_aixs_tuser) ,  // 输出帧开始信号
    .O_tdata   (m_aixs_tdata) ,  // 输出数据
    .O_tvalid  (m_aixs_tvalid) ,  // 输出数据有效信号
    .O_tready    (m_aixs_tready)    // 输出数据准备好信号
);

data128_96 u_data128_96 (
    .I_tdata(m_aixs_tdata),
    .O_tdata(m_aixs_tdata_96)
);

awb #(
    .IMG_HEIGHT(720 ),
    .IMG_WIDTH (1280)
) u_awb (
    .I_clk   (axi4s_video_aclk),
    .I_rst_n (I_rst_n),
    .I_frame_bad(I_frame_bad),
    .I_tlast (m_aixs_tlast),
    .I_tuser (m_aixs_tuser),
    .I_tdata (m_aixs_tdata_96),
    .I_tvalid(m_aixs_tvalid),
    .I_tready(m_aixs_tready),
    .O_tlast (awb_O_tlast ),
    .O_tuser (awb_O_tuser ),
    .O_tdata (awb_O_tdata ),
```

```
    .O_tvalid(awb_O_tvalid),
    .O_tready(awb_O_tready)
);

    // ---- 饱和度增强级（2026-10-04 新增：AWB 之后、128bit 重排之前）----
    // 背景：整条链只有 demosaic+AWB，无 CCM/饱和度；画面发灰。
    // 2026-10-04 第二轮：K_Q4 21->28（1.31x->1.75x，用户反馈"还不够"）；
    // 参数位宽扩至 6bit（上限 3.94x）。整链仿真位精确对拍已通过（2x921600 像素）。
    wire        sat_O_tlast, sat_O_tuser, sat_O_tvalid;
    wire [95:0] sat_O_tdata;

    axis_saturation_rgb96 #(
        .K_Q4(6'd28)
    ) u_saturation (
        .I_clk  (axi4s_video_aclk),
        .I_rst_n(I_rst_n),
        .I_tlast(awb_O_tlast ),
        .I_tuser(awb_O_tuser ),
        .I_tdata(awb_O_tdata ),
        .I_tvalid(awb_O_tvalid),
        .O_tlast(sat_O_tlast ),
        .O_tuser(sat_O_tuser ),
        .O_tdata(sat_O_tdata ),
        .O_tvalid(sat_O_tvalid)
    );

    data_96bit_to_128bit u_data_96bit_to_128bit(
        .I_clk              ( axi4s_video_aclk  ),
        .I_rst_n            ( I_rst_n           ),

        .I_96b_frame_start  ( sat_O_tuser       ),
        .I_96b_valid        ( sat_O_tvalid      ),
        .I_96b_tlast        ( sat_O_tlast       ),
        .I_96b_data         ( sat_O_tdata       ),

        .O_128b_frame_start ( awb_O_tuser_128   ),
        .O_128b_tlast       ( repack_O_tlast    ),
        .O_128b_valid       ( awb_O_tvalid_128  ),
        .O_128b_data        ( awb_O_tdata_128   )
    );



//data96_128 u_data96_128 (
//    .I_tdata(awb_O_tdata),
//    .O_tdata(awb_O_tdata_128)
//);

endmodule


// Four-pixel RGB888 gamma stage.  The 16-segment curve approximates
// y = 255 * (x / 255)^0.7.  It lifts shadows and mid-tones while preserving
// black and white and adds one registered pipeline cycle.
module axis_gamma_rgb96 (
    input  wire        I_clk,
    input  wire        I_rst_n,
    input  wire        I_tlast,
    input  wire        I_tuser,
    input  wire [95:0] I_tdata,
    input  wire        I_tvalid,
    output reg         O_tlast,
    output reg         O_tuser,
```

```
    output reg  [95:0] O_tdata,
    output reg         O_tvalid
);

    function [7:0] gamma07;
        input [7:0] value;
        reg [7:0] base;
        reg [7:0] delta;
        reg [11:0] interpolation;
        begin
            case (value[7:4])
                4'h0: begin base = 8'd0;   delta = 8'd37; end
                4'h1: begin base = 8'd37;  delta = 8'd23; end
                4'h2: begin base = 8'd60;  delta = 8'd19; end
                4'h3: begin base = 8'd79;  delta = 8'd18; end
                4'h4: begin base = 8'd97;  delta = 8'd16; end
                4'h5: begin base = 8'd113; delta = 8'd16; end
                4'h6: begin base = 8'd129; delta = 8'd14; end
                4'h7: begin base = 8'd143; delta = 8'd14; end
                4'h8: begin base = 8'd157; delta = 8'd14; end
                4'h9: begin base = 8'd171; delta = 8'd13; end
                4'ha: begin base = 8'd184; delta = 8'd13; end
                4'hb: begin base = 8'd197; delta = 8'd12; end
                4'hc: begin base = 8'd209; delta = 8'd12; end
                4'hd: begin base = 8'd221; delta = 8'd12; end
                4'he: begin base = 8'd233; delta = 8'd11; end
                default: begin base = 8'd244; delta = 8'd11; end
            endcase

            interpolation = delta * value[3:0] + 8;
            if (value == 8'hff)
                gamma07 = 8'hff;
            else
                gamma07 = base + (interpolation >> 4);
        end
    endfunction

    always @(posedge I_clk or negedge I_rst_n) begin
        if (!I_rst_n) begin
            O_tlast  <= 1'b0;
            O_tuser  <= 1'b0;
            O_tdata  <= 96'd0;
            O_tvalid <= 1'b0;
        end else begin
            O_tlast  <= I_tlast;
            O_tuser  <= I_tuser;
            O_tvalid <= I_tvalid;
            if (I_tvalid) begin
                O_tdata <= {
                    gamma07(I_tdata[95:88]), gamma07(I_tdata[87:80]), gamma07(I_tdata[79:72]),
                    gamma07(I_tdata[71:64]), gamma07(I_tdata[63:56]), gamma07(I_tdata[55:48]),
                    gamma07(I_tdata[47:40]), gamma07(I_tdata[39:32]), gamma07(I_tdata[31:24]),
                    gamma07(I_tdata[23:16]), gamma07(I_tdata[15:8]),  gamma07(I_tdata[7:0])
                };
            end else begin
                O_tdata <= 96'd0;
            end
        end
    end

endmodule
```

```
// Centre crop for the ISP's four-pixel-wide RGB888 stream.  Frame start is
// intentionally forwarded at the original frame boundary so video_in can
// reset its asynchronous FIFO before the first selected row arrives.
module axis_crop_rgb96 #(
    parameter integer IN_WIDTH   = 1792,
    parameter integer IN_HEIGHT  = 1080,
    parameter integer OUT_WIDTH  = 1600,
    parameter integer OUT_HEIGHT = 900,
    parameter integer X_START    = 96,
    parameter integer Y_START    = 90
)(
    input  wire        I_clk,
    input  wire        I_rst_n,
    input  wire        I_tlast,
    input  wire        I_tuser,
    input  wire [95:0] I_tdata,
    input  wire        I_tvalid,
    output wire        O_tlast,
    output wire        O_tuser,
    output wire [95:0] O_tdata,
    output wire        O_tvalid
);

    localparam integer X_START_BEAT = X_START / 4;
    localparam integer OUT_BEATS    = OUT_WIDTH / 4;
    localparam integer X_END_BEAT   = X_START_BEAT + OUT_BEATS;

    reg [9:0]  S_hcnt;
    reg [10:0] S_vcnt;

    wire [9:0]  S_current_h = I_tuser ? 10'd0 : S_hcnt;
    wire [10:0] S_current_v = I_tuser ? 11'd0 : S_vcnt;
    wire S_inside = (S_current_h >= X_START_BEAT) &&
                    (S_current_h < X_END_BEAT) &&
                    (S_current_v >= Y_START) &&
                    (S_current_v < (Y_START + OUT_HEIGHT));

    assign O_tdata  = I_tdata;
    assign O_tvalid = I_tvalid && S_inside;
    assign O_tlast  = O_tvalid && (S_current_h == (X_END_BEAT - 1));
    assign O_tuser  = I_tuser;

    always @(posedge I_clk or negedge I_rst_n) begin
        if(!I_rst_n) begin
            S_hcnt <= 10'd0;
            S_vcnt <= 11'd0;
        end
        else if(I_tuser) begin
            S_hcnt <= (I_tvalid && !I_tlast) ? 10'd1 : 10'd0;
            S_vcnt <= (I_tvalid && I_tlast) ? 11'd1 : 11'd0;
        end
        else if(I_tvalid) begin
            if(I_tlast) begin
                S_hcnt <= 10'd0;
                S_vcnt <= (S_vcnt == IN_HEIGHT - 1) ? 11'd0 : S_vcnt + 1'b1;
            end
            else begin
                S_hcnt <= S_hcnt + 1'b1;
            end
        end
    end

endmodule
```

```
// ============================================================================


// ============================================================================
// axis_saturation_rgb96.v —— RGB 饱和度增强级（4 像素/拍，96-bit 流）
// ----------------------------------------------------------------------------
// 背景：板上画面"发灰、饱和度低"的根因是 ISP 链路只有 demosaic + AWB，
//       没有色彩校正（CCM）也没有饱和度处理；本模块补一个最小可用的饱和度级。
//
// 公式（全整数，与 tools/sat_tool.py 严格一致，可位精确对拍）：
//     Y   = (77*R + 150*G + 29*B) >> 8      // BT.601 亮度近似，8bit
//     d   = ch - Y                          // 有符号
//     out = clamp(Y + ((K_Q4 * d + 8) >> 4))// Q4.4 增益，16=1.00x, 28≈1.75x, 32=2.00x
//
// 接口风格与 isp_top 内 axis_gamma_rgb96 一致：单时钟、无背压、2 拍流水；
// tvalid/tlast/tuser 同步延迟 2 拍，与数据严格对齐（帧边界不受影响）。
//
// 字节序约定：与 AWB 输出一致 —— 每 24bit 一个像素 {R[23:16], G[15:8], B[7:0]}，
//             [23:0]=第 0 像素 … [95:72]=第 3 像素；模块逐通道原位变换，不改布局。
// ============================================================================
module axis_saturation_rgb96 #(
    parameter [5:0] K_Q4 = 6'd21          // Q4.4 增益: 16=1.00x(恒等), 21≈1.31x, 28≈1.75x, 32=2.00x, 上限
 63≈3.94x
)(
    input  wire        I_clk,
    input  wire        I_rst_n,
    input  wire        I_tlast,
    input  wire        I_tuser,
    input  wire [95:0] I_tdata,
    input  wire        I_tvalid,
    output reg         O_tlast,
    output reg         O_tuser,
    output reg  [95:0] O_tdata,
    output reg         O_tvalid
);

    // ---------------- 第一级：每像素亮度 Y + 原色寄存 ----------------
    reg [7:0] s1_c [0:11];                // 4 像素 x {R,G,B}：3*i+0=R, +1=G, +2=B
    reg [7:0] s1_y [0:3];
    reg       s1_valid, s1_last, s1_user;
    integer   i;

    always @(posedge I_clk or negedge I_rst_n) begin
        if (!I_rst_n) begin
            s1_valid <= 1'b0;
            s1_last  <= 1'b0;
            s1_user  <= 1'b0;
        end else begin
            s1_valid <= I_tvalid;
            s1_last  <= I_tlast;
            s1_user  <= I_tuser;
            for (i = 0; i < 4; i = i + 1) begin
                s1_c[3*i+0] <= I_tdata[24*i + 23 -: 8];   // R
                s1_c[3*i+1] <= I_tdata[24*i + 15 -: 8];   // G
                s1_c[3*i+2] <= I_tdata[24*i +  7 -: 8];   // B
                s1_y[i]     <= (77  * I_tdata[24*i + 23 -: 8]
                              + 150 * I_tdata[24*i + 15 -: 8]
                              + 29  * I_tdata[24*i +  7 -: 8]) >> 8;
            end
        end
    end
```

```
    // ---------------- 第二级：饱和增益 + 限幅 ----------------
    function [7:0] sat_ch;
        input [7:0] c;
        input [7:0] y;
        reg signed [8:0]  d;
        reg signed [15:0] t;
        reg signed [11:0] v;                    // 12bit：K≤63 满量程（评审#7：原10bit 在 K≥33 时先于限幅溢出
)
        begin
            d = $signed({1'b0, c}) - $signed({1'b0, y});
            t = d * $signed({1'b0, K_Q4});        // 9bit x 7bit -> 16bit(signed)，|d|·K≤255×63=16065
            t = t + 16'sd8;                        // 四舍五入(+0.5)
            v = $signed({1'b0, y}) + (t >>> 4);    // 算术右移 = floor，与 Python >> 一致
            if (v < 0)          sat_ch = 8'd0;
            else if (v > 16'sd255) sat_ch = 8'd255;
            else                sat_ch = v[7:0];
        end
    endfunction

    reg [7:0] s2_o [0:11];
    reg       s2_valid, s2_last, s2_user;
    integer   j;

    always @(posedge I_clk or negedge I_rst_n) begin
        if (!I_rst_n) begin
            s2_valid <= 1'b0;
            s2_last  <= 1'b0;
            s2_user  <= 1'b0;
        end else begin
            s2_valid <= s1_valid;
            s2_last  <= s1_last;
            s2_user  <= s1_user;
            for (j = 0; j < 4; j = j + 1) begin
                s2_o[3*j+0] <= sat_ch(s1_c[3*j+0], s1_y[j]);
                s2_o[3*j+1] <= sat_ch(s1_c[3*j+1], s1_y[j]);
                s2_o[3*j+2] <= sat_ch(s1_c[3*j+2], s1_y[j]);
            end
        end
    end

    // ---------------- 输出寄存（总延迟 2 拍） ----------------
    always @(posedge I_clk or negedge I_rst_n) begin
        if (!I_rst_n) begin
            O_tvalid <= 1'b0;
            O_tlast  <= 1'b0;
            O_tuser  <= 1'b0;
            O_tdata  <= 96'd0;
        end else begin
            O_tvalid <= s2_valid;
            O_tlast  <= s2_last;
            O_tuser  <= s2_user;
            O_tdata  <= { s2_o[ 9], s2_o[10], s2_o[11],   // 像素3 (95:72)
                          s2_o[ 6], s2_o[ 7], s2_o[ 8],   // 像素2 (71:48)
                          s2_o[ 3], s2_o[ 4], s2_o[ 5],   // 像素1 (47:24)
                          s2_o[ 0], s2_o[ 1], s2_o[ 2] }; // 像素0 (23:0)
        end
    end

endmodule
```

5.4 tb_awb.v —— AWB 单元测试台（含第2批边界相位；新增）

直接实例化工程版 awb.v（板上真值）。38 项检查全绿（PASS）：正常链（复位/首步/两帧滞后/帧内恒定/限幅 3.0x/帧中突变）+ 边界（20480 拒/20481 采、亮白与近黑剔除、超长帧拒、坏帧门拒+恢复、帧中复位）。

（247 行，MD5 6b224750）

```
`timescale 1ns/1ps
// ============================================================================
// tb_awb.v —— AWB 帧同步化行为验证（评审#1 · 第1批，2026-10-04）
// ----------------------------------------------------------------------------
// 被测：工程版 awb.v（lab_ex1 树 = 板上真值），本次改动：
//   (1) 上一帧统计锁存（仅当帧 > STAT_MIN_BEATS 非白拍才采信）
//   (2) 增益 = clamp(上一帧统计商, 0.5x~3.0x)，帧内恒定（不再随累加漂移）
//   (3) 帧边界按 ±0.125x/帧 步进（跨帧平滑）；不再每帧复位 1.0
//
// 激励：合成强色偏像素流，帧长 NF=24000 拍（> 20480 阈值），行 320 拍。
// 说明（期望值推导）：统计初值 +1 → 目标商偶有 1LSB 偏差（如 1.5x 实测 0x17FFF），
//   容差 TOL=0x100 覆盖；限幅测试用 3.75x 比值（若 clamp 失效会到 0x31FFF+，必判错）。
//   A 相 C1=(R100,G150,B200)：R 目标 1.5x、B 目标 0.75x — 两帧滞后后逐帧 ±0.125
//   B 相 C2=(R40, G150,B200)：R 比值 3.75x 触发**限幅** → 精确停在 3.0x(0x30000)
//   C 相 帧中突变（第 12000 拍切色）：当帧增益恒定，下一帧也不变（目标滞后）
// 判据：全部 CHECK OK → TB_RESULT: PASS
// ============================================================================
module tb_awb;
    reg clk = 0, rst_n = 0;
    always #7 clk = ~clk;                       // ~71.4MHz

    localparam NF  = 24000;                     // 拍/帧（> STAT_MIN_BEATS=20480）
    localparam LN  = 320;                       // 行内拍数（1280px / 4px）
    localparam GAP = 200;                       // 帧间空拍
    localparam [19:0] TOL = 20'h00100;          // 容差 ±1/256（吸收统计初值+1的舍入）

    reg        v = 0;
    reg [95:0] d = 0;
    reg        tuser = 0, tlast = 0;
    reg [7:0]  cr = 8'd100, cg = 8'd150, cb = 8'd200;

    wire [23:0] px = {cr, cg, cb};
    always @* d = {px, px, px, px};

    reg bad = 0;                                // 第2批：I_frame_bad 驱动（帧首呈现"上一帧坏"）

    awb #(.IMG_HEIGHT(720), .IMG_WIDTH(1280)) u_awb (
        .I_clk(clk), .I_rst_n(rst_n),
        .I_tlast(tlast), .I_tuser(tuser), .I_tdata(d), .I_tvalid(v),
        .I_frame_bad(bad),
        .I_tready(),
        .O_tlast(), .O_tuser(), .O_tvalid(), .O_tdata(),
        .O_tready(1'b1)
    );

    integer fails = 0;
    reg [19:0] gr, gb;                          // 帧头采样
    reg [19:0] gr_end, gb_end;                  // 帧尾采样

    task frame_head(input [7:0] r, input [7:0] g, input [7:0] b);
        begin
            @(posedge clk);
            tuser <= 1'b1; v <= 1'b1; tlast <= 1'b0;
            cr <= r; cg <= g; cb <= b;
            @(posedge clk);
```

```
            tuser <= 1'b0;
            repeat (7) @(posedge clk);      // 头段共 8 拍（tuser 拍 + 7）→ 整帧拍数精确 = 调用值
            gr = u_awb.in_r_r1;
            gb = u_awb.in_b_r1;
        end
    endtask

    task frame_body(input integer sw_at,
                    input [7:0] r1, input [7:0] g1, input [7:0] b1,
                    input integer end_sample);
        integer i;
        begin
            for (i = 8; i < NF; i = i + 1) begin
                v     <= 1'b1;
                tlast <= ((i % LN) == (LN - 1));
                if (sw_at > 0 && i == sw_at) begin cr <= r1; cg <= g1; cb <= b1; end
                @(posedge clk);
            end
            if (end_sample) begin
                gr_end = u_awb.in_r_r1;
                gb_end = u_awb.in_b_r1;
            end
            v <= 1'b0; tlast <= 1'b0;
            repeat (GAP) @(posedge clk);
        end
    endtask

    task chk(input [8*40:1] tag, input [19:0] er, input [19:0] eb);
        reg [19:0] dr, db;
        begin
            dr = (gr > er) ? gr - er : er - gr;
            db = (gb > eb) ? gb - eb : eb - gb;
            if (dr > TOL || db > TOL) begin
                $display("CHECK FAIL [%0s]: gain_R=%h (exp %h)  gain_B=%h (exp %h)", tag, gr, er, gb, eb)
;
                fails = fails + 1;
            end else
                $display("CHECK OK   [%0s]: gain_R=%h  gain_B=%h", tag, gr, gb);
        end
    endtask

    task chk_eq(input [8*40:1] tag, input [19:0] vr, input [19:0] vb);
        begin
            if ((gr_end !== vr) || (gb_end !== vb)) begin
                $display("CHECK FAIL [%0s]: intra-frame drift R %h->%h  B %h->%h", tag, vr, gr_end, vb, g
b_end);
                fails = fails + 1;
            end else
                $display("CHECK OK   [%0s]: intra-frame constant R=%h B=%h", tag, vr, vb);
        end
    endtask

    // ---- 第2批：可变帧长版本（边界相位用）----
    task bhead(input [7:0] r, input [7:0] g, input [7:0] b, input integer fb);
        begin
            @(posedge clk);
            bad   <= fb;                        // 本帧首呈现的"上一帧坏"标志
            tuser <= 1'b1; v <= 1'b1; tlast <= 1'b0;
            cr <= r; cg <= g; cb <= b;
            @(posedge clk);
            tuser <= 1'b0;
            repeat (7) @(posedge clk);      // 头段共 8 拍（tuser 拍 + 7）→ 整帧拍数精确 = 调用值
            gr = u_awb.in_r_r1;
```

```
            gb = u_awb.in_b_r1;
        end
    endtask

    task bbody(input integer n);
        integer i;
        begin
            for (i = 8; i < n; i = i + 1) begin
                v     <= 1'b1;
                tlast <= ((i % 320) == 319);
                @(posedge clk);
            end
            v <= 1'b0; tlast <= 1'b0;
            repeat (GAP) @(posedge clk);
        end
    endtask

    task breset;
        begin
            rst_n = 0; repeat (20) @(posedge clk);
            rst_n = 1; repeat (10) @(posedge clk);
        end
    endtask

    initial begin
        rst_n = 0;
        repeat (20) @(posedge clk);
        rst_n = 1;
        repeat (10) @(posedge clk);

        // ============ Phase A: C1=(100,150,200) -> R target 1.5x, B target 0.75x ============
        frame_head(100,150,200); chk("F01 reset 1.0", 20'h10000, 20'h10000);          frame_body(0,0,0,0,
0);
        frame_head(100,150,200); chk("F02 no-prev still 1.0", 20'h10000, 20'h10000);  frame_body(0,0,0,0,
0);
        frame_head(100,150,200); chk("F03 first step +-0.125", 20'h12000, 20'h0E000); frame_body(0,0,0,0,
0);
        frame_head(100,150,200); chk("F04 B hits 0.75", 20'h14000, 20'h0C000);        frame_body(0,0,0,0,
0);
        frame_head(100,150,200); chk("F05", 20'h16000, 20'h0C000);                    frame_body(0,0,0,0,
0);
        frame_head(100,150,200); chk("F06 R hits ~1.5", 20'h17FFF, 20'h0C000);        frame_body(0,0,0,0,
1);
        chk_eq("F06 intra-frame constant", gr, gb);
        frame_head(100,150,200); chk("F07 steady", 20'h17FFF, 20'h0C000);             frame_body(0,0,0,0,
0);
        frame_head(100,150,200); chk("F08 steady", 20'h17FFF, 20'h0C000);             frame_body(0,0,0,0,
0);

        // ============ Phase B: C2=(40,150,200) -> ratio 3.75x, CLAMPED to 3.0x ============
        frame_head(40,150,200);  chk("F09 lag-2 frames (hold)", 20'h17FFF, 20'h0C000); frame_body(0,0,0,0
,0);
        frame_head(40,150,200);  chk("F10 lag-1 frame (hold)", 20'h17FFF, 20'h0C000);  frame_body(0,0,0,0
,0);
        frame_head(40,150,200);  chk("F11 climb", 20'h19FFF, 20'h0C000);               frame_body(0,0,0,0
,0);
        frame_head(40,150,200);  chk("F12", 20'h1BFFF, 20'h0C000);                     frame_body(0,0,0,0
,0);
        frame_head(40,150,200);  chk("F13", 20'h1DFFF, 20'h0C000);                     frame_body(0,0,0,0
,0);
        frame_head(40,150,200);  chk("F14", 20'h1FFFF, 20'h0C000);                     frame_body(0,0,0,0
,0);
        frame_head(40,150,200);  chk("F15", 20'h21FFF, 20'h0C000);                     frame_body(0,0,0,0
```

```
,0);
        frame_head(40,150,200);  chk("F16", 20'h23FFF, 20'h0C000);                    frame_body(0,0,0,0
,0);
        frame_head(40,150,200);  chk("F17", 20'h25FFF, 20'h0C000);                    frame_body(0,0,0,0
,0);
        frame_head(40,150,200);  chk("F18", 20'h27FFF, 20'h0C000);                    frame_body(0,0,0,0
,0);
        frame_head(40,150,200);  chk("F19", 20'h29FFF, 20'h0C000);                    frame_body(0,0,0,0
,0);
        frame_head(40,150,200);  chk("F20", 20'h2BFFF, 20'h0C000);                    frame_body(0,0,0,0
,0);
        frame_head(40,150,200);  chk("F21", 20'h2DFFF, 20'h0C000);                    frame_body(0,0,0,0
,0);
        frame_head(40,150,200);  chk("F22", 20'h2FFFF, 20'h0C000);                    frame_body(0,0,0,0
,0);
        frame_head(40,150,200);  chk("F23 CLAMP -> exactly 3.0x", 20'h30000, 20'h0C000); frame_body(0,0,0
,0,0);
        frame_head(40,150,200);  chk("F24 clamp hold", 20'h30000, 20'h0C000);         frame_body(0,0,0,0
,0);

        // ============ Phase C: mid-frame switch (no response in-frame / next frame) ============
        frame_head(40,150,200);  chk("F25 before switch", 20'h30000, 20'h0C000);
        frame_body(12000, 100,150,200, 1);                    // beat 12000: switch back to C1
        chk_eq("F25 intra-frame constant (switch ignored)", gr, gb);
        frame_head(100,150,200); chk("F26 lag -> hold 3.0x", 20'h30000, 20'h0C000);    frame_body(0,0,0,0
,0);

        // ================= 第2批边界相位（评审批次(2)：窗口/暗剔除/坏帧门）=================
        // G1 下界 20480（不达标→拒；若误采信，C 帧头会显示 3.0x 方向步进 0x12000）
        breset;
        bhead(50,150,200,0); bbody(20480);
        bhead(100,150,200,0); bbody(24000);
        bhead(100,150,200,0); chk("G1 min-1 rejected", 20'h10000, 20'h10000); bbody(24000);
        bhead(100,150,200,0); chk("G1 next step ok", 20'h12000, 20'h0E000);  bbody(24000);

        // G2 下界 20481（达标→采信，体现在 C' 帧头 0x12000/0xE000）
        breset;
        bhead(50,150,200,0); bbody(20481);
        bhead(100,150,200,0); bbody(24000);
        bhead(100,150,200,0); chk("G2 min+1 committed", 20'h12000, 20'h0E000); bbody(24000);

        // H 高亮非中性（255,220,255 sum=730>720 → 剔除；若误计会向下步进）
        breset;
        bhead(255,220,255,0); bbody(24000);
        bhead(100,150,200,0); bbody(24000);
        bhead(100,150,200,0); chk("H bright excluded", 20'h10000, 20'h10000); bbody(24000);

        // H2 近黑非中性（5,15,20 sum=40<45 → 新下界剔除；若误计 R 目标=3.0 会向上步进）
        breset;
        bhead(5,15,20,0); bbody(24000);
        bhead(100,150,200,0); bbody(24000);
        bhead(100,150,200,0); chk("H2 dark excluded", 20'h10000, 20'h10000); bbody(24000);
        // H2 对照：过下界 (10,30,40) sum=80 → 采信
        breset;
        bhead(10,30,40,0); bbody(24000);
        bhead(100,150,200,0); bbody(24000);
        bhead(100,150,200,0); chk("H2 dark-control ok", 20'h12000, 20'h0E000); bbody(24000);

        // J 超长帧上界（310000>300000 → 拒；对应 720p 漏 SOF 并帧 460800 的保护）
        breset;
        bhead(50,150,200,0); bbody(310000);
        bhead(100,150,200,0); bbody(24000);
        bhead(100,150,200,0); chk("J oversize rejected", 20'h10000, 20'h10000); bbody(24000);
```

```
        // K frame_bad 门（长度合格但"上一帧坏" → 拒；下一帧恢复采信）
        breset;
        bhead(50,150,200,0); bbody(24000);                                    // K1 好统计
        bhead(100,150,200,1); bbody(24000);                                   // K2 帧首 fb=1 → 评估 K1 时
拒
        bhead(100,150,200,0); chk("K bad-frame rejected", 20'h10000, 20'h10000); bbody(24000);
        bhead(100,150,200,0); chk("K gate recovers", 20'h12000, 20'h0E000);   bbody(24000);

        // L 帧中复位 → 增益回 1.0
        bhead(50,150,200,0); bbody(2000);
        rst_n = 0; repeat (5) @(posedge clk); rst_n = 1; repeat (10) @(posedge clk);
        gr = u_awb.in_r_r1;
        gb = u_awb.in_b_r1;
        chk("L reset-mid-frame", 20'h10000, 20'h10000);

        $display("============================================================");
        if (fails == 0)
            $display("TB_RESULT: PASS  (0 fail)  AWB frame-sync behavior confirmed");
        else
            $display("TB_RESULT: FAIL  (fails=%0d)", fails);
        $finish;
    end

    initial begin
        #200000000;    // watchdog: 200ms sim time (actual ~9ms)
        $display("TB_RESULT: FAIL (watchdog timeout)");
        $finish;
    end
endmodule
```

5.5 tb_video_passthrough.v —— 整链仿真 TB（含饱和度 SAT_EN 开关）

虚拟 RAW10 传感器 → ISP(明码) → 饱和度级（TB 内插入，SAT_EN/SAT_K_Q4 可切换）→ 弹性串化 → 逐帧转储。位精确对拍即基于其转储帧。

（182 行，MD5 579747d2）

```
`timescale 1ns/1ps
// ============================================================================
// tb_video_passthrough.v —— 视频传输路线整链仿真（摄像头画面到达验证）
// ----------------------------------------------------------------------------
// 链路：虚拟 RAW10 传感器(文件回放) -> isp_top(官方明码: demosaic/AWB/重排)
//       -> [饱和度增强 axis_saturation_rgb96 (SAT_EN=1 时，本次新增)]
//       -> 96bit 抽头 -> ser96to24(1ppc) -> frame_dump24(逐帧转储)
//
// 判据与输出：
//   1. 转储 passthrough_frame_N.hex（每帧 1280x720 RGB888，行主序）
//      —— 这就是"屏幕应该显示的画面"（仿真里的 HDMI 替身）
//   2. 几何报告：AWB 输出的 tuser/tlast 计数（诚实打印；
//      已知发现：demosaic 的 tuser 链在纯 RTL 仿真可能不激活，
//      上板若"有数据无帧同步"先查这里 —— 见使用指导 PDF）
//   3. PASS = 至少 2 帧完整转储
//   饱和度级说明：SAT_EN=1 时抽头取饱和度输出（K_Q4 见下）；
//   SAT_EN=0 可复现未加饱和度级的基线帧（与 baseline_oct03/ 对拍）。
// ============================================================================
module tb_video_passthrough;
    reg clk = 0, rst_n = 0;
    always #7 clk = ~clk;                 // ~71.4MHz（axi4s_video_aclk 仿真值）

    // ---------------- 虚拟 RAW10 传感器（文件回放） ----------------
    reg [39:0] beats [0:921599];          // 40bit = 4 像素 x RAW10
    reg [1:0]  metas [0:921599];          // bit1=sof(帧首) bit0=eol(行尾)
```

```
integer    nb;
integer    bi = 0;

initial begin
    $readmemh("raw10_beats.hex", beats);
    $readmemh("raw10_meta.txt",  metas);
    nb = 230400 * 3;                  // 3 帧馈送: 前 2 帧为验收目标, 第 3 帧兼作流水冲刷
end

wire        isp_ready;                // isp 对上游的就绪指示
reg         raw_valid = 1'b0;
reg  [39:0] raw_data  = 40'd0;
reg         raw_sof = 1'b0, raw_eol = 1'b0;

// 馈送占空 1/8：真实 MIPI 传感器按节拍出图，且下游串化器每 4 拍才
// 消费 1 个 beat——满速馈送会淹没 1ppc 消费端（溢出丢像素）。
// 1/8 占空馈送 + 正确 AXI 握手（每拍恰好被消费一次）：
// - 满 1/8：demosaic 输出 4px/拍 vs 转储侧 1px/拍存在速率差，
//   满速会让弹性 FIFO 满而搁浅尾帧；1/8 占空留足排空余量。
// - 历史：valid 跨拍保持会使同一 beat 被重复消费 8 次，eol/tuser
//   计数 x8 打乱 Bayer 行相位——务必一拍一消费。
reg [2:0] pres;
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) pres <= 3'd0;
    else        pres <= pres + 3'd1;
end
wire feed_slot = (pres == 3'd0);
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        raw_valid <= 1'b0;
    end else if (raw_valid) begin
        if (isp_ready) begin
            raw_valid <= 1'b0;                // 本拍已被消费
        end
    end else if (feed_slot && (bi < nb)) begin
        raw_data  <= beats[bi];
        raw_sof   <= metas[bi][1];
        raw_eol   <= metas[bi][0];
        raw_valid <= 1'b1;
        bi = bi + 1;
    end
end

// ---------------- 官方 ISP 明码链 ----------------
wire [127:0] isp_tdata;
wire         isp_tvalid, isp_tuser, isp_tlast;

isp_top u_isp (
    .axi4s_video_aclk (clk),
    .I_rst_n          (rst_n),
    .I_tlast          (raw_eol),
    .I_tuser          (raw_sof),
    .I_tdata          (raw_data),
    .I_tvalid         (raw_valid),
    .I_tdest          (10'd0),
    .O_tready         (isp_ready),     // 下游就绪 -> ISP 上游流控
    .O_tdata          (isp_tdata),
    .O_tlast          (isp_tlast),
    .O_tuser          (isp_tuser),
    .O_tvalid         (isp_tvalid),
    .I_tready         ()               // 输出端口, 仿真不关心
);
```

```
// ---------------- 96bit 抽头（AWB 输出）-> 弹性串化 -> 逐帧转储 ----------------
// 说明：官方链路 ISP 输出送 DDR 帧缓存（深缓冲吸收行级突发）；
// 仿真没有 DDR，用 px_serializer_fifo（8192 beat 弹性 FIFO + 真反压）
// 充当"浅 DDR"，保证零丢拍。ser96to24 是无反压窃流设计（13 深 FIFO），
// 吸不住 demosaic 的行级突发，不能用在整链仿真里。
// ---------------- 饱和度增强级（本次新增） ----------------
// SAT_EN=1：抽头取饱和度输出；SAT_EN=0：旁路（复现基线）。
localparam       SAT_EN   = 1'b1;
localparam [5:0] SAT_K_Q4 = 6'd28;      // 16=1.00x, 21≈1.31x, 28≈1.75x

wire        sat_tvalid, sat_tuser, sat_tlast;
wire [95:0] sat_tdata;

axis_saturation_rgb96 #(.K_Q4(SAT_K_Q4)) u_sat (
    .I_clk  (clk), .I_rst_n(rst_n),
    .I_tlast(u_isp.awb_O_tlast),  .I_tuser(u_isp.awb_O_tuser),
    .I_tdata(u_isp.awb_O_tdata),  .I_tvalid(u_isp.awb_O_tvalid),
    .O_tlast(sat_tlast), .O_tuser(sat_tuser),
    .O_tdata(sat_tdata), .O_tvalid(sat_tvalid)
);

wire        tap_valid = SAT_EN ? sat_tvalid : u_isp.awb_O_tvalid;
wire        tap_sof   = SAT_EN ? sat_tuser : u_isp.awb_O_tuser;
wire        tap_eol   = SAT_EN ? sat_tlast : u_isp.awb_O_tlast;
wire [95:0] tap_data  = SAT_EN ? sat_tdata : u_isp.awb_O_tdata;

wire        ser_valid, ser_sof, ser_eol;
wire [23:0] ser_data;
wire [15:0] ser_ovf;

px_serializer_fifo u_ser (
    .I_clk   (clk), .I_rst_n(rst_n),
    .I_valid (tap_valid),                  // 128->96 重排（+饱和度）后的 4 像素/拍
    .I_sof   (tap_sof),
    .I_eol   (tap_eol),
    .I_data  (tap_data),
    .O_ready (isp_ready),                  // 真反压回 ISP 上游
    .O_valid (ser_valid), .O_sof(ser_sof),
    .O_eol   (ser_eol),   .O_data(ser_data),
    .O_dbg_overflows(ser_ovf)
);

wire dump_done;
frame_dump24 #(
    .BASE_NAME ("passthrough_frame"),
    .MAX_FRAMES(3)
) u_dump (
    .I_clk(clk), .I_rst_n(rst_n),
    .I_valid(ser_valid), .I_eol(ser_eol), .I_data(ser_data),
    .O_frame_done(dump_done)
);

// ---------------- 几何观察（诚实打印，不作 PASS 依据） ----------------
integer tuser_total = 0, tlast_total = 0;
always @(posedge clk) begin
    if (rst_n && u_isp.awb_O_tvalid) begin
        if (u_isp.awb_O_tuser) tuser_total = tuser_total + 1;
        if (u_isp.awb_O_tlast) tlast_total = tlast_total + 1;
    end
end

// ---------------- 终判 ----------------
integer dumped = 0;
```

```
    always @(posedge clk) begin
        if (rst_n && dump_done) dumped = dumped + 1;
    end

    initial begin
        repeat (5) @(posedge clk);
        rst_n = 1;
    end

    initial begin
        #500000000;   // 看门狗：1/8 馈送约 77ms + 行窗节流，500ms 足够 3 帧
        $display("TB_RESULT: %s frames_dumped=%0d tuser_total=%0d tlast_total=%0d ser_ovf=%0d",
                 (dumped >= 2) ? "PASS" : "FAIL", dumped, tuser_total, tlast_total, ser_ovf);
        if (tuser_total == 0)
            $display("NOTE: tuser 链在本纯 RTL 仿真环境不激活（已知发现，见使用指导 PDF 第 5 节）");
        $finish;
    end

    // 提前收工：3 帧全部转储完
    always @(posedge clk) begin
        if (rst_n && dumped >= 3) begin
            $display("TB_RESULT: PASS frames_dumped=%0d tuser_total=%0d tlast_total=%0d ser_ovf=%0d",
                     dumped, tuser_total, tlast_total, ser_ovf);
            if (tuser_total == 0)
                $display("NOTE: tuser 链在本纯 RTL 仿真环境不激活（已知发现，见使用指导 PDF 第 5 节）");
            $finish;
        end
    end

endmodule
```

5.6 px_serializer_fifo.v —— 仿真用“浅 DDR”（弹性 FIFO + 真反压）

整链仿真里替代 DDR 帧缓存的角色：4 像素/拍 → 1 像素/拍。

（83 行，MD5 `1fe244bf`）

```
`timescale 1ns/1ps
// ============================================================================
// px_serializer_fifo.v — 仿真专用弹性串化器（96bit 4像素/拍 -> 1像素/拍）
// ----------------------------------------------------------------------------
// 与 ser96to24（无反压窃流设计）不同：本模块带 8192 beat 弹性 FIFO，
// 并把"空位不足"如实反压回上游（O_ready -> isp 上游流控）。
// 官方链路里 ISP 输出去往 DDR 帧缓存（深缓冲吸收行级突发）；
// 仿真里没有 DDR，用本模块充当"浅 DDR"，保证零丢拍。
// 每个存储字 = {eol, sof, data[95:0]}；sof 标在首像素、eol 标在第 4 像素。
// 仅供仿真，禁止综合。
// ============================================================================
module px_serializer_fifo #(
    parameter DEPTH = 8192                    // beat 数（2 的幂）
)(
    input  wire        I_clk,
    input  wire        I_rst_n,
    input  wire        I_valid,
    input  wire        I_sof,
    input  wire        I_eol,
    input  wire [95:0] I_data,
    output wire        O_ready,               // 对上游的反压（空位充足才为 1）
    output reg         O_valid,
    output reg         O_sof,
    output reg         O_eol,
    output reg  [23:0] O_data,
```

```
    output reg  [15:0] O_dbg_overflows        // 调试：上游被反压的拍数（I_valid 且 !O_ready）；
                                              // 本设计上游尊重 ready，故不代表丢拍数
);
    localparam AW = $clog2(DEPTH);            // 地址位宽（随 DEPTH 泛化）
    reg [97:0] mem [0:DEPTH-1];
    reg [AW-1:0] waddr, raddr;
    reg [AW:0]   count;                       // FIFO 内 beat 数（多 1 位以容纳满值）
    reg [1:0]  px_sel;                        // 当前 beat 内像素指针

    assign O_ready = (count < DEPTH - 4);

    wire wr        = I_valid && O_ready;
    wire rd        = (count != 14'd0);
    wire beat_done = rd && (px_sel == 2'd3);  // 本拍第 4 像素已输出 = beat 消费完

    wire [97:0] rword = mem[raddr];
    wire [23:0] rpx   = (px_sel == 2'd0) ? rword[23:0]   :
                        (px_sel == 2'd1) ? rword[47:24]  :
                        (px_sel == 2'd2) ? rword[71:48]  :
                                           rword[95:72];

    always @(posedge I_clk or negedge I_rst_n) begin
        if (!I_rst_n) begin
            waddr <= 13'd0; raddr <= 13'd0; count <= 14'd0; px_sel <= 2'd0;
            O_valid <= 1'b0; O_sof <= 1'b0; O_eol <= 1'b0; O_data <= 24'd0;
            O_dbg_overflows <= 16'd0;
        end else begin
            O_valid <= 1'b0;

            if (wr) begin
                mem[waddr] <= {I_eol, I_sof, I_data};
                waddr <= waddr + 13'd1;
            end
            if (beat_done)
                raddr <= raddr + 13'd1;

            // count: 写 +1，beat 消费完 -1（同拍两者相抵）
            if (wr && !beat_done)        count <= count + 14'd1;
            else if (!wr && beat_done)   count <= count - 14'd1;

            if (rd) begin
                O_valid <= 1'b1;
                O_data  <= rpx;
                O_sof   <= rword[96] && (px_sel == 2'd0);   // 首像素携带 sof
                O_eol   <= rword[97] && (px_sel == 2'd3);   // 末像素携带 eol
                px_sel <= (px_sel == 2'd3) ? 2'd0 : px_sel + 2'd1;
            end else begin
                O_sof <= 1'b0;
                O_eol <= 1'b0;
            end

            if (I_valid && !O_ready)
                O_dbg_overflows <= O_dbg_overflows + 16'd1;
        end
    end

endmodule
```

5.7 烧录脚本（交付包内三个一键 bat）

burn_camera_sram.bat（原版直通流）/ burn_camera_sat_sram.bat（当前定版，见 6.4）/ burn_colorbar_sram.bat（彩条自检）。要点：自动确保 hwserver 在跑——手动启动必须带 **-r**（remote-bitbang，端口 5555；不带只开设备不开服务——本次实测教训）。

burn_camera_sram.bat（MD5 `5c409c61`）：

```
@echo off
REM ========================================================================
REM  Headless SRAM burn: camera_to_dsi_display.bit（原版直通流）
REM  前置：TD6.2.1 位于 D:\TD6.2.1；FT2232 A 通道已装 WinUSB；
REM        hwserver 手动启动必须带 -r（本脚本自动处理）。
REM ========================================================================
setlocal
set "BIT=%~dp0camera_to_dsi_display.bit"
set "TD_BIN=D:\TD6.2.1\bin"
set "TCL=%TEMP%\bw_burn_sram.tcl"
set "OUT=%TEMP%\bw_burn_sram_out.txt"

if not exist "%BIT%" ( echo [ERROR] bit file not found: %BIT% & pause & exit /b 1 )

echo [INFO] bit file : %BIT%
certutil -hashfile "%BIT%" MD5

netstat -ano | findstr ":5555" | findstr LISTENING >nul
if errorlevel 1 (
  echo [INFO] no hwserver on port 5555, starting one...
  start "" "%TD_BIN%\hwserver.exe" -r
  timeout /t 3 /nobreak >nul
  netstat -ano | findstr ":5555" | findstr LISTENING >nul
  if errorlevel 1 ( echo [ERROR] hwserver failed to start - check cable/driver & pause & exit /b 1 )
)

>  "%TCL%" echo connect_cable -host_name 127.0.0.1:5555 -open
>> "%TCL%" echo download -bit "%BIT%"

"%TD_BIN%\bw_commands_prompt.exe" "%TCL%" > "%OUT%" 2>&1
type "%OUT%"
findstr /C:"PRG-2014" "%OUT%" >nul
if errorlevel 1 ( echo. & echo [ERROR] burn FAILED (no chip-validation line above) & pause & exit /b 1 )

echo.
echo [DONE] Look at the display. (SRAM load is lost on power-off.)
pause
```

burn_camera_sat_sram.bat（MD5 `32bd2e09`）：

```
@echo off
REM ========================================================================
REM  Headless SRAM burn: 饱和度增强(K28≈1.75x) + AWB帧同步化(第1批)
REM                      + 接口修复(第2批：repack tlast/异常帧门) 版比特流
REM  构建溯源：08_board_bringup/03_td_rebuild_sat/lab_ex1_build（syn→phy 两段无头流程）
REM  验收记录：见仓库 04_awb_framesync / 05_isp_interface（单元 TB 全绿）
REM ========================================================================
setlocal
set "BIT=%~dp0camera_to_dsi_display_sat_k28_awbfs_b2.bit"
set "TD_BIN=D:\TD6.2.1\bin"
set "TCL=%TEMP%\bw_burn_sat.tcl"
set "OUT=%TEMP%\bw_burn_sat_out.txt"

if not exist "%BIT%" ( echo [ERROR] bit file not found: %BIT% & pause & exit /b 1 )

echo [INFO] bit file : %BIT%
```

```
certutil -hashfile "%BIT%" MD5

netstat -ano | findstr ":5555" | findstr LISTENING >nul
if errorlevel 1 (
  echo [INFO] no hwserver on port 5555, starting one...
  start "" "%TD_BIN%\hwserver.exe" -r
  timeout /t 3 /nobreak >nul
  netstat -ano | findstr ":5555" | findstr LISTENING >nul
  if errorlevel 1 ( echo [ERROR] hwserver failed to start - check cable/driver & pause & exit /b 1 )
)

>  "%TCL%" echo connect_cable -host_name 127.0.0.1:5555 -open
>> "%TCL%" echo download -bit "%BIT%"

"%TD_BIN%\bw_commands_prompt.exe" "%TCL%" > "%OUT%" 2>&1
type "%OUT%"
findstr /C:"PRG-2014" "%OUT%" >nul
if errorlevel 1 ( echo. & echo [ERROR] burn FAILED (no chip-validation line above) & pause & exit /b 1 )

echo.
echo [DONE] Look at the display: colors should be noticeably more saturated
echo        than the original bit, and color should be stable (no frame-top flash).
pause
```

burn_colorbar_sram.bat（MD5 `9851e339`）：

```
@echo off
REM ========================================================================
REM  Headless SRAM burn: diag_hdmi_colorbar_1280x720.bit（彩条自检）
REM  用途：只验证"板子→显示器"链路（不带摄像头因素）。
REM ========================================================================
setlocal
set "BIT=%~dp0diag_hdmi_colorbar_1280x720.bit"
set "TD_BIN=D:\TD6.2.1\bin"
set "TCL=%TEMP%\bw_burn_colorbar.tcl"
set "OUT=%TEMP%\bw_burn_colorbar_out.txt"

if not exist "%BIT%" ( echo [ERROR] bit file not found: %BIT% & pause & exit /b 1 )

echo [INFO] bit file : %BIT%
certutil -hashfile "%BIT%" MD5

netstat -ano | findstr ":5555" | findstr LISTENING >nul
if errorlevel 1 (
  echo [INFO] no hwserver on port 5555, starting one...
  start "" "%TD_BIN%\hwserver.exe" -r
  timeout /t 3 /nobreak >nul
  netstat -ano | findstr ":5555" | findstr LISTENING >nul
  if errorlevel 1 ( echo [ERROR] hwserver failed to start - check cable/driver & pause & exit /b 1 )
)

>  "%TCL%" echo connect_cable -host_name 127.0.0.1:5555 -open
>> "%TCL%" echo download -bit "%BIT%"

"%TD_BIN%\bw_commands_prompt.exe" "%TCL%" > "%OUT%" 2>&1
type "%OUT%"
findstr /C:"PRG-2014" "%OUT%" >nul
if errorlevel 1 ( echo. & echo [ERROR] burn FAILED (no chip-validation line above) & pause & exit /b 1 )

echo.
echo [DONE] Colorbar pattern should be on the display now.
pause
```

5.8 tools/sat_tool.py —— 饱和度预览/位精确对拍工具（新增）

sweep：多档 K 预览拼图（仿真帧 + 实拍照片示意）；check：RTL 转储帧 vs Python 同式参考逐字节比对（与 RTL 全整数同式，含算术右移 floor 语义）。check 模式已严格帧尺寸校验。

（174 行，MD5 `666f3a5e`）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""sat_tool.py — 饱和度增强工具（预览 / RTL 位精确校验）

公式（与 RTL axis_saturation_rgb96.v 严格一致，全整数）：
    Y   = (77*R + 150*G + 29*B) >> 8          # BT.601, 8bit
    d   = c - Y                               # 有符号
    t   = (K_Q4 * d + 8) >> 4                 # K_Q4: Q4.4, 16=1.00x, 21≈1.31x
    out = clamp(Y + t, 0, 255)

子命令：
  sweep  — 对一帧 hex 做多档 K 预览拼图（+可选实拍照片示意）
  check  — 校验 RTL 仿真输出帧与 Python 参考是否逐字节一致

用法：
  python sat_tool.py sweep --frame passthrough_frame_0.hex --out sat_sweep_sim.png \
      --photo D:/QQ/IMG_20261004_171700.jpg --photo-out sat_sweep_photo.png
  python sat_tool.py check --before baseline.hex --after new.hex --k 21
"""
import argparse
import sys

W, H = 1280, 720


def sat_pixel(r, g, b, k_q4):
    y = (77 * r + 150 * g + 29 * b) >> 8
    out = []
    for c in (r, g, b):
        v = y + ((k_q4 * (c - y) + 8) >> 4)
        out.append(0 if v < 0 else (255 if v > 255 else v))
    return out[0], out[1], out[2]


def load_hex(path):
    px = []
    with open(path, "r") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            v = int(line, 16)
            px.append(((v >> 16) & 0xFF, (v >> 8) & 0xFF, v & 0xFF))
    if len(px) != W * H:
        print(f"[warn] {path}: {len(px)} pixels != {W*H}")
    return px


def apply_k(px, k_q4):
    tv = (16 * 77) | (16 * 150) | (16 * 29)  # unused, keep lint quiet
    return [sat_pixel(r, g, b, k_q4) for (r, g, b) in px]


# ---------------- sweep ----------------

def cmd_sweep(args):
    from PIL import Image, ImageDraw, ImageFont
```

```
def font(size, bold=False):
    for p in (r"C:\Windows\Fonts\msyhbd.ttc" if bold else r"C:\Windows\Fonts\msyh.ttc",
              r"C:\Windows\Fonts\arialbd.ttf" if bold else r"C:\Windows\Fonts\arial.ttf"):
        try:
            return ImageFont.truetype(p, size)
        except Exception:
            continue
    return ImageFont.load_default()

ks = [16, 21, 28, 32]
labels = ["original (K=16, x1.00)", "K=21 (x1.31)", "K=28 (x1.75)", "K=32 (x2.00)"]

px = load_hex(args.frame)
im0 = Image.new("RGB", (W, H))
im0.putdata(px)

# 以红方块为中心裁一块，放大看差异
box = (400, 225, 880, 495)  # 480x270
panels = [im0.crop(box)]
for k in ks[1:]:
    tmp = Image.new("RGB", (W, H))
    tmp.putdata(apply_k(px, k))
    panels.append(tmp.crop(box))

pw, ph = panels[0].size
scale = 2
f = font(20, bold=True)
lab_h = 34
canvas = Image.new("RGB", (pw * 2 * scale + 30, (ph * scale + lab_h) * 2 + 30), (24, 24, 24))
d = ImageDraw.Draw(canvas)
for i, (p, lab) in enumerate(zip(panels, labels)):
    p = p.resize((pw * scale, ph * scale), Image.NEAREST)
    col, row = i % 2, i // 2
    x = 10 + col * (pw * scale + 10)
    y = 10 + row * (ph * scale + lab_h + 10)
    canvas.paste(p, (x, y + lab_h))
    d.text((x + 6, y + 4), lab, fill=(240, 240, 240), font=f)
canvas.save(args.out)
print(f"[ok] 仿真帧对比图 -> {args.out}")

if args.photo:
    ph_im = Image.open(args.photo).convert("RGB")
    w0, h0 = ph_im.size
    # 取中下部桌面区域（有彩色物体的地方）做横构图
    band = ph_im.crop((0, int(h0 * 0.50), w0, int(h0 * 0.95)))
    band = band.resize((760, int(band.height * 760 / band.width)))
    pv = [band] + [None] * 3
    # 用像素级变换（缩略图尺寸，速度OK）
    for idx, k in enumerate(ks[1:], start=1):
        tmp = band.copy()
        data = [sat_pixel(r, g, b, k) for (r, g, b) in list(tmp.getdata())]
        tmp.putdata(data)
        pv[idx] = tmp
    bw, bh = band.size
    c2 = Image.new("RGB", (bw * 2 + 30, (bh + lab_h) * 2 + 30), (24, 24, 24))
    d2 = ImageDraw.Draw(c2)
    for i, (p, lab) in enumerate(zip(pv, labels)):
        col, row = i % 2, i // 2
        x = 10 + col * (bw + 10)
        y = 10 + row * (bh + lab_h + 10)
        c2.paste(p, (x, y + lab_h))
        d2.text((x + 6, y + 4), lab + "  [实拍照片示意]", fill=(240, 240, 240), font=font(20, True))
```

```
        c2.save(args.photo_out)
        print(f"[ok] 实拍照片示意 -> {args.photo_out}")


# ---------------- check ----------------

def cmd_check(args):
    before = load_hex(args.before)
    after = load_hex(args.after)
    if len(before) != W * H or len(after) != W * H:
        # check 模式严格：任一侧非 1280x720 直接 FAIL（sweep 允许 warn）
        print(f"[FAIL] 帧尺寸非 {W}x{H}: before={len(before)} after={len(after)}")
        sys.exit(1)
    bad = 0
    first = None
    for i, ((r, g, b), (R, G, B)) in enumerate(zip(before, after)):
        e = sat_pixel(r, g, b, args.k)
        if (R, G, B) != e:
            bad += 1
            if first is None:
                first = (i, (r, g, b), (R, G, B), e)
    n = len(before)
    if bad == 0:
        print(f"[PASS] {n} 像素全部与 Python 参考一致 (K_Q4={args.k})")
    else:
        y, x = first[0] // W, first[0] % W
        print(f"[FAIL] {bad}/{n} 像素不一致; 首个: ({x},{y}) in={first[1]} rtl={first[2]} py={first[3]}")
        sys.exit(1)


def main():
    ap = argparse.ArgumentParser()
    sub = ap.add_subparsers(dest="cmd", required=True)

    sp = sub.add_parser("sweep")
    sp.add_argument("--frame", required=True)
    sp.add_argument("--out", required=True)
    sp.add_argument("--photo")
    sp.add_argument("--photo-out", default="sat_sweep_photo.png")
    sp.set_defaults(func=cmd_sweep)

    cp = sub.add_parser("check")
    cp.add_argument("--before", required=True)
    cp.add_argument("--after", required=True)
    cp.add_argument("--k", type=int, default=21)
    cp.set_defaults(func=cmd_check)

    args = ap.parse_args()
    args.func(args)


if __name__ == "__main__":
    main()
```

5.9 data_96bit_to_128bit —— 96→128 重排（第2批重构版，板用副本全文）

第2批改动：(1) 气泡保相位（valid=0 不再清 S_cnt，仅帧首复位；评审#8）；(2) tlast 在 repack 内重建（O_128b_tlast 与 O_128b_valid/O_128b_data 同拍；评审#4/#5）。行长为 4 拍整数倍（720p 行 320 拍）时 tlast 恒在相位 3。本文件另含未用的 data96_128 模块。

（119 行，MD5 fde22863）

```
module data_96bit_to_128bit (
    input wire        I_clk,
    input wire        I_rst_n,

    input wire        I_96b_frame_start,
    input wire        I_96b_valid,
    input wire        I_96b_tlast,          // 评审#4/#8（第2批）：行尾，与 I_96b_valid 同拍
    input wire[95:0]  I_96b_data,

    output reg        O_128b_frame_start,
    output reg        O_128b_tlast,         // 与 O_128b_valid/O_128b_data 同拍重建
    output reg        O_128b_valid,
    output reg[127:0] O_128b_data
);



    reg[95:0]  S_96b_data_1d;
    reg        S_96b_valid_1d;
    reg[1:0]   S_cnt;


    always @(posedge I_clk) begin
        S_96b_valid_1d <= I_96b_valid;
        S_96b_data_1d  <= I_96b_data;

O_128b_frame_start <= I_96b_frame_start;
    end


    // 相位：仅在有效拍前进、气泡保持（评审#8）；帧首强制相位 0
    // 注意：行长为 4 拍整数倍（720p 行 320 拍）时 tlast 落在相位 3；行长非 4 倍时
    // 相位 0 拍的 tlast 会被抑制（无对应输出字）。
    wire[1:0]  cnt_eff = I_96b_frame_start ? 2'd0 : S_cnt;

    always @(posedge I_clk) begin
        if(I_96b_frame_start && I_96b_valid)
            S_cnt <= 2'd1;                      // 帧首拍按相位 0 处理
        else if(I_96b_frame_start)
            S_cnt <= 2'd0;
        else if(I_96b_valid)
            S_cnt <= S_cnt + 2'd1;
        // 气泡（valid=0）：保持相位
    end


    always @(posedge I_clk) begin
        if(I_96b_valid)
            begin
                case(cnt_eff)
                    'd0 :
                        begin
                            O_128b_valid <= 1'b0;
                            O_128b_tlast <= 1'b0;
                            O_128b_data  <= 'd0;
                        end
                    'd1 :
                        begin
                            O_128b_valid <= 1'b1;
                            O_128b_tlast <= I_96b_tlast;
```

```
                        O_128b_data  <= {S_96b_data_1d,I_96b_data[95:64]};
                    end
                'd2 :
                    begin
                        O_128b_valid <= 1'b1;
                        O_128b_tlast <= I_96b_tlast;
                        O_128b_data  <= {S_96b_data_1d[63:0],I_96b_data[95:32]};
                    end
                'd3 :
                    begin
                        O_128b_valid <= 1'b1;
                        O_128b_tlast <= I_96b_tlast;
                        O_128b_data  <= {S_96b_data_1d[31:0],I_96b_data};
                    end
                default:
                    begin
                        O_128b_valid <= 1'b0;
                        O_128b_tlast <= 1'b0;
                        O_128b_data  <= 'd0;
                    end
            endcase
        end
    else
        begin
            O_128b_valid <= 1'b0;
            O_128b_tlast <= 1'b0;
            O_128b_data  <= 'd0;
        end
end



endmodule









/*
module data96_128 (

    input wire [95:0] I_tdata,

    output wire [127:0] O_tdata

);

  assign O_tdata = {
    {8'd0}, I_tdata[95:72], {8'd0}, I_tdata[71:48], {8'd0}, I_tdata[47:24], {8'd0}, I_tdata[23:0]
  };

endmodule
*/
```

5.10 tb_repack.v —— post-repack 记分板（第2批(1)，新增）

byte-stream 记分板 + 气泡注入（4 个相位各一次）+ tlast 落位 + back-to-back 帧；结果见 6.6（全部 CHECK OK）。

（187 行，MD5 c0f3f773）

```
`timescale 1ns/1ps
// ============================================================================
// tb_repack.v —— data_96bit_to_128bit 重构验证（第2批(1)，2026-10-05）
// ----------------------------------------------------------------------------
// 被测：板用副本 isp/data96_128/data96_128.v 内的 data_96bit_to_128bit
// 改动：(1) 气泡保相位（valid=0 不再清 S_cnt，仅帧首复位）(2) tlast 在 repack 内重建
//
// 判据（全部为可核验的字节级性质）：
//  P1 无气泡：输出 128b 字流 == 输入 96b 比特流按 128b 重切（byte-stream scoreboard）
//  P2 气泡在 4 个相位各注一次：输出字流与 P1 逐字一致（相位保持的直接证据）
//  P3 tlast：每行末拍的 tlast 恰好落在包含该行最后一比特的输出字序号上
//  P4 back-to-back：帧首强制相位 0，第二帧字流独立正确（无第一帧残留污染）
// ============================================================================
module tb_repack;
    reg clk = 0, rst_n = 0;
    always #7 clk = ~clk;                       // ~71.4MHz

    localparam ROW_BEATS = 8;                   // 本 TB 行 = 8 拍（4 的整数倍；720p 实为 320）
    localparam NROWS     = 4;                   // 帧 = 32 拍 = 3072 bit = 24 × 128b
    localparam NBEATS    = ROW_BEATS * NROWS;   // 32
    localparam NWORDS    = NBEATS * 3 / 4;      // 24
    localparam GAP       = 24;                  // 帧间空拍

    reg         v = 0, tuser = 0, tlast = 0;
    reg  [95:0] d = 0;

    wire         o_fs, o_tl, o_v;
    wire [127:0] o_d;

    data_96bit_to_128bit u_dut (
        .I_clk(clk), .I_rst_n(rst_n),
        .I_96b_frame_start(tuser), .I_96b_valid(v),
        .I_96b_tlast(tlast), .I_96b_data(d),
        .O_128b_frame_start(o_fs), .O_128b_tlast(o_tl),
        .O_128b_valid(o_v), .O_128b_data(o_d)
    );

    integer fails = 0;

    // ---- 采集 ----
    reg [127:0] got [0:255];
    integer     got_n;
    integer     tl_idx [0:15];
    integer     tl_n;

    always @(posedge clk) begin
        if (o_v) begin
            got[got_n] <= o_d;
            if (o_tl) begin
                tl_idx[tl_n] <= got_n;
                tl_n <= tl_n + 1;
            end
            got_n <= got_n + 1;
        end
    end

    // ---- 激励 ----
    reg [95:0] exp_beats [0:255];

    task drive_frame(input [31:0] base, input integer gap_at, input integer gap_len);
        integer k;
        begin
```

```
            @(posedge clk);
            tuser <= 1'b1; v <= 1'b1; tlast <= 1'b0;
            d     <= {32'hA0000000 + base, 32'hB0000000 + base, 32'hC0000000 + base};
            exp_beats[0] = {32'hA0000000 + base, 32'hB0000000 + base, 32'hC0000000 + base};
            @(posedge clk);
            tuser <= 1'b0;
            for (k = 1; k < NBEATS; k = k + 1) begin
                v     <= 1'b1;
                d     <= {32'hA0000000 + base + k, 32'hB0000000 + base + k, 32'hC0000000 + base + k};
                exp_beats[k] = {32'hA0000000 + base + k, 32'hB0000000 + base + k, 32'hC0000000 + base + k
};
                tlast <= ((k % ROW_BEATS) == (ROW_BEATS - 1));
                @(posedge clk);
                if (gap_at > 0 && k == gap_at) begin          // 气泡：valid 拉低 gap_len 拍
                    v <= 1'b0; tlast <= 1'b0;
                    repeat (gap_len) @(posedge clk);
                end
            end
            v <= 1'b0; tlast <= 1'b0;
            repeat (GAP) @(posedge clk);
        end
    endtask

    // ---- 记分板：期望字 = 输入比特流按 128b 重切 ----
    reg [3071:0] frame_bits;
    reg [127:0]  exp_word;
    task scoreboard(input [8*24:1] tag, input [31:0] base);
        integer k, j, bad_cnt;
        begin
            // 组帧比特流：beat k 占 [96k +: 96]
            for (k = 0; k < NBEATS; k = k + 1)
                frame_bits[(NBEATS-1-k)*96 +: 96] = exp_beats[k];   // beat0 在流的高位（先发送）
            bad_cnt = 0;
            if (got_n != NWORDS) begin
                $display("CHECK FAIL [%0s]: 输出字数 %0d != 期望 %0d", tag, got_n, NWORDS);
                fails = fails + 1;
            end
            for (j = 0; j < NWORDS; j = j + 1) begin
                exp_word = frame_bits[128*(NWORDS-1-j) +: 128];    // 第 j 个字 = 流的第 j 段（自高向低）
                if (got[j] !== exp_word) begin
                    if (bad_cnt < 3)
                        $display("CHECK FAIL [%0s]: word[%0d] got=%h exp=%h", tag, j, got[j], exp_word);
                    bad_cnt = bad_cnt + 1;
                end
            end
            if (bad_cnt == 0 && got_n == NWORDS)
                $display("CHECK OK   [%0s]: wordstream identical (%0d words)", tag, NWORDS);
            else if (bad_cnt > 0)
                fails = fails + 1;

            // tlast 位置：行 r 末拍 k=8r+7，末比特 96k+95 → 字序号 (96k+95)/128 = 6r+5
            if (tl_n != NROWS) begin
                $display("CHECK FAIL [%0s]: tlast 次数 %0d != %0d", tag, tl_n, NROWS);
                fails = fails + 1;
            end else begin
                bad_cnt = 0;
                for (j = 0; j < NROWS; j = j + 1)
                    if (tl_idx[j] != (3 * ROW_BEATS / 4) * j + (3 * ROW_BEATS / 4 - 1))
                        bad_cnt = bad_cnt + 1;
                if (bad_cnt == 0)
                    $display("CHECK OK   [%0s]: tlast at row-end words {5,11,17,23}", tag);
                else begin
                    $display("CHECK FAIL [%0s]: tlast idx %0d/%0d/%0d/%0d (exp 5/11/17/23)",
```

```
                            tag, tl_idx[0], tl_idx[1], tl_idx[2], tl_idx[3]);
                    fails = fails + 1;
                end
            end
        end
    endtask

    task clear_capture;
        begin
            got_n = 0; tl_n = 0;
        end
    endtask

    initial begin
        rst_n = 0;
        repeat (20) @(posedge clk);
        rst_n = 1;
        repeat (10) @(posedge clk);

        // ---- P1: 无气泡 ----
        clear_capture;
        drive_frame(32'd0, 0, 0);
        scoreboard("P1 no-bubble", 32'd0);

        // ---- P2: 气泡注入在 4 个相位（k mod 4 = 1/2/3/0 之后）----
        clear_capture;
        drive_frame(32'h100, 1, 3);          // 气泡出现在 beat1 之后 → 相位断点 (k mod 4)=1
        scoreboard("P2 bubble@phase1", 32'h100);

        clear_capture;
        drive_frame(32'h200, 2, 3);          // 相位断点 =2
        scoreboard("P2 bubble@phase2", 32'h200);

        clear_capture;
        drive_frame(32'h300, 3, 5);          // 相位断点 =3
        scoreboard("P2 bubble@phase3", 32'h300);

        clear_capture;
        drive_frame(32'h400, 4, 7);          // 相位断点 =0（帧首组内）
        scoreboard("P2 bubble@phase0", 32'h400);

        // ---- P4: back-to-back（第二帧紧跟第一帧，数据模式不同）----
        clear_capture;
        drive_frame(32'h500, 0, 1);          // 帧1（GAP=1 拍，几乎背靠背）
        clear_capture;                        // 清采集，只评帧2
        drive_frame(32'h777, 2, 2);          // 帧2 带气泡
        scoreboard("P4 back-to-back frame2", 32'h777);

        $display("============================================================");
        if (fails == 0)
            $display("TB_RESULT: PASS  (0 fail)  repack hold-phase + tlast rebuild verified");
        else
            $display("TB_RESULT: FAIL  (fails=%0d)", fails);
        $finish;
    end

    initial begin
        #50000000;   // 看门狗 50ms
        $display("TB_RESULT: FAIL (watchdog timeout)");
        $finish;
    end
endmodule
```

5.11 编译清单（整链仿真 / AWB 单元 / repack 单元）

filelist_sim.f（整链）：

```
# filelist_sim.f —— 视频通路整链仿真编译清单（ModelSim 用）
# 约定：D:/fpga-ptz-tracker 为仓库根；本清单供 msim_passthrough.do 使用
# 注意：全部为仿真专用文件，禁止加入 TD 工程

# ---- 视觉方向（串化器 + 帧转储） ----
D:/fpga-ptz-tracker/03_vision/step1_video_link/03_project/rtl/ser96to24.v
D:/fpga-ptz-tracker/03_vision/step1_video_link/03_project/rtl/frame_dump24.v

# ---- 饱和度增强级（本次新增；ISP 链内插在 AWB 之后） ----
D:/fpga-ptz-tracker/04_embedded/03_project/rtl/axis_saturation_rgb96.v

# ---- 官方 ISP 明码副本（04_embedded/sim）----
D:/fpga-ptz-tracker/04_embedded/03_project/sim/isp_top.v
D:/fpga-ptz-tracker/04_embedded/03_project/sim/data128_96.v
D:/fpga-ptz-tracker/04_embedded/03_project/sim/data_96bit_to_128bit.v
D:/fpga-ptz-tracker/04_embedded/03_project/sim/demosaic_4x_2_0/demosaic.v
D:/fpga-ptz-tracker/04_embedded/03_project/sim/demosaic_4x_2_0/zhenghe.v
D:/fpga-ptz-tracker/04_embedded/03_project/sim/demosaic_4x_2_0/bilinear_interpolation.v
D:/fpga-ptz-tracker/04_embedded/03_project/sim/demosaic_4x_2_0/line_buffer_demosaic.v
D:/fpga-ptz-tracker/04_embedded/03_project/sim/demosaic_4x_2_0/mipi_to_raw_converter.v
D:/fpga-ptz-tracker/04_embedded/03_project/sim/demosaic_4x_2_0/raw_matrix_3x3_buffer.v
D:/fpga-ptz-tracker/04_embedded/03_project/sim/demosaic_4x_2_0/blk_mem_gen_demosaic/blk_mem_gen_demosaic.
v
D:/fpga-ptz-tracker/04_embedded/03_project/sim/demosaic_4x_2_0/blk_mem_gen_demosaic/RTL/ram_c36c2ca3b3a7.
v
D:/fpga-ptz-tracker/04_embedded/03_project/sim/demosaic_4x_2_0/blk_mem_gen_zhenghe/blk_mem_gen_zhenghe.v
D:/fpga-ptz-tracker/04_embedded/03_project/sim/demosaic_4x_2_0/blk_mem_gen_zhenghe/RTL/ram_d32970204f32.v
D:/fpga-ptz-tracker/04_embedded/03_project/sim/awb.v
D:/fpga-ptz-tracker/04_embedded/03_project/sim/awb/signal_delay.v
D:/fpga-ptz-tracker/04_embedded/03_project/sim/awb/ram/blk_mem_gen_awb_delay_signal.v

# ---- 行为级替身（TD 工程禁止加入）----
D:/fpga-ptz-tracker/04_embedded/03_project/rtl/divider_gate_sim.v
D:/fpga-ptz-tracker/04_embedded/03_project/rtl/ph1p_logic_eram_stub.v

# ---- 本 TB ----
D:/fpga-ptz-tracker/08_board_bringup/01_video_passthrough/sim/px_serializer_fifo.v
D:/fpga-ptz-tracker/08_board_bringup/01_video_passthrough/sim/tb_video_passthrough.v
```

filelist_awb_tb.f（单元）：

```
# tb_awb 单元测试编译清单（2026-10-04 第1批：AWB 帧同步化）
# 被测 = 工程版 awb.v（板上真值）；signal_delay/BRAM 用仿真树副本；divider 用行为替身
D:/fpga-ptz-tracker/08_board_bringup/01_video_passthrough/lab_ex1/user_source/hdl_source/awb.v
D:/fpga-ptz-tracker/04_embedded/03_project/sim/awb/signal_delay.v
D:/fpga-ptz-tracker/04_embedded/03_project/sim/awb/ram/blk_mem_gen_awb_delay_signal.v
D:/fpga-ptz-tracker/04_embedded/03_project/rtl/divider_gate_sim.v
D:/fpga-ptz-tracker/04_embedded/03_project/rtl/ph1p_logic_eram_stub.v
D:/fpga-ptz-tracker/08_board_bringup/04_awb_framesync/tb_awb.v
```

filelist_tb_repack.f（repack 单元）：

```
# tb_repack 编译清单（2026-10-05 第2批(1)：repack 保相位 + tlast 重建）
D:/fpga-ptz-tracker/08_board_bringup/01_video_passthrough/lab_ex1/user_source/hdl_source/isp/data96_128/d
ata96_128.v
D:/fpga-ptz-tracker/08_board_bringup/05_isp_interface/tb_repack.v
```

六、验证证据

本章为全部验证证据的原始记录；日志原件随仓库归档（路径见各小节标题）。

6.1 AWB 单元 TB 结果（28 项全绿，原始日志已归档 04_awb_framesync/tb_awb_run.log）

```
# CHECK OK   [F01 reset 1.0]: gain_R=10000  gain_B=10000
# CHECK OK   [F02 no-prev still 1.0]: gain_R=10000  gain_B=10000
# CHECK OK   [F03 first step +-0.125]: gain_R=12000  gain_B=0e000
# CHECK OK   [F04 B hits 0.75]: gain_R=14000  gain_B=0c000
# CHECK OK   [F05]: gain_R=16000  gain_B=0c000
# CHECK OK   [F06 R hits ~1.5]: gain_R=17fff  gain_B=0c000
# CHECK OK   [F06 intra-frame constant]: intra-frame constant R=17fff B=0c000
# CHECK OK   [F07 steady]: gain_R=17fff  gain_B=0c000
# CHECK OK   [F08 steady]: gain_R=17fff  gain_B=0c000
# CHECK OK   [F09 lag-2 frames (hold)]: gain_R=17fff  gain_B=0c000
# CHECK OK   [F10 lag-1 frame (hold)]: gain_R=17fff  gain_B=0c000
# CHECK OK   [F11 climb]: gain_R=19fff  gain_B=0c000
# CHECK OK   [F12]: gain_R=1bfff  gain_B=0c000
# CHECK OK   [F13]: gain_R=1dfff  gain_B=0c000
# CHECK OK   [F14]: gain_R=1ffff  gain_B=0c000
# CHECK OK   [F15]: gain_R=21fff  gain_B=0c000
# CHECK OK   [F16]: gain_R=23fff  gain_B=0c000
# CHECK OK   [F17]: gain_R=25fff  gain_B=0c000
# CHECK OK   [F18]: gain_R=27fff  gain_B=0c000
# CHECK OK   [F19]: gain_R=29fff  gain_B=0c000
# CHECK OK   [F20]: gain_R=2bfff  gain_B=0c000
# CHECK OK   [F21]: gain_R=2dfff  gain_B=0c000
# CHECK OK   [F22]: gain_R=2ffff  gain_B=0c000
# CHECK OK   [F23 CLAMP -> exactly 3.0x]: gain_R=30000  gain_B=0c000
# CHECK OK   [F24 clamp hold]: gain_R=30000  gain_B=0c000
# CHECK OK   [F25 before switch]: gain_R=30000  gain_B=0c000
# CHECK OK   [25 intra-frame constant (switch ignored)]: intra-frame constant R=30000 B=0c000
# CHECK OK   [F26 lag -> hold 3.0x]: gain_R=30000  gain_B=0c000
# CHECK OK   [G1 min-1 rejected]: gain_R=10000  gain_B=10000
# CHECK OK   [G1 next step ok]: gain_R=12000  gain_B=0e000
# CHECK OK   [G2 min+1 committed]: gain_R=12000  gain_B=0e000
# CHECK OK   [H bright excluded]: gain_R=10000  gain_B=10000
# CHECK OK   [H2 dark excluded]: gain_R=10000  gain_B=10000
# CHECK OK   [H2 dark-control ok]: gain_R=12000  gain_B=0e000
# CHECK OK   [J oversize rejected]: gain_R=10000  gain_B=10000
# CHECK OK   [K bad-frame rejected]: gain_R=10000  gain_B=10000
# CHECK OK   [K gate recovers]: gain_R=12000  gain_B=0e000
# CHECK OK   [L reset-mid-frame]: gain_R=10000  gain_B=10000
# TB_RESULT: PASS  (0 fail)  AWB frame-sync behavior confirmed
```

期望值推导说明：统计初值 +1 使目标商偶有 1LSB 偏差（1.5x 实测 0x17FFF，容差 TOL=0x100 覆盖）；限幅检查特意用 3.75x 比值——若 clamp 失效增益会爬到 0x31FFF+（远超容差必判错），实测精确停在 0x30000。

6.2 整链仿真与位精确对拍

```
TB_RESULT: PASS frames_dumped=2 tuser_total=3 tlast_total=2008 ser_ovf=0      （K=28 与 K=50 两轮同值）
sat_tool check K=28: [PASS] 921600 像素全部与 Python 参考一致（帧0/帧1 各一次）
sat_tool check K=50: [PASS] 921600 像素全部与 Python 参考一致（评审#7 修复后的溢出场景）
```

注：AWB 帧同步化并入工程版后，整链"位精确对拍"谱系由未并入的仿真变体 **AWB** 保持（见 6.1 说明与 7.3 风险 R3）；AWB 新行为由单元 TB 对板上真值验证。

6.3 时序证据（摘自各代 phy 日志 final 段）

```
K21 : Setup WNS  255ps  TNS 0ps  NUM_FEPS 0 | Hold WNS 20ps  TNS 0ps
K28 : Setup WNS  703ps  TNS 0ps  NUM_FEPS 0 | Hold WNS 20ps  TNS 0ps
K28+AWB(awbfs,第1批): Setup WNS 176ps  TNS 0ps  NUM_FEPS 0 | Hold WNS 20ps  TNS 0ps
K28+AWB+接口(awbfs_b2,第2批): Setup WNS 143ps  TNS 0ps  NUM_FEPS 0 | Hold WNS 20ps  TNS 0ps
```

6.4 烧录实测记录（无头 CLI，hwserver -r / 端口 5555）

记录归属（如实标注）：以下为 `…_sat_k28.bit`（曾定版）的完整烧录实录。当前定版 `…_awbfs_b2.bit` 已于 **2026-10-05** 烧录实测成功（同判据：Chip validation success + 进度 100% + 返回 0，见本节末补录）；色彩稳定性由现场目视确认。

```
connect_cable -host_name 127.0.0.1:5555 -open   → 0.005s, 返回 0
read_device_id                                  → PH1P35MDG324
download -bit …/camera_to_dsi_display_sat_k28.bit
  PRG-2014 : Chip validation success: PH1P35MDG324
  0%…100% 进度条走满（约 12s），命令返回 0

补录：awbfs_b2（当前定版，第1+2批全部改动）
  download -bit …/camera_to_dsi_display_sat_k28_awbfs_b2.bit
  PRG-2014 : Chip validation success: PH1P35MDG324  → 返回 0   (2026-10-05)
```

6.5 源码一致性核验（本轮收尾时实测）

```
三处 isp_top.v          MD5 一致（含 u_saturation + repack tlast 接入 + I_frame_bad）
三处 awb.v              MD5 一致（含 #1 帧同步化 + 第2批窗口/暗剔除/frame_bad 门）
三处 design_top_wrapper MD5 一致（含 S_frame_bad 接线）
三处 data96_128.v       MD5 一致（含保相位 + tlast 重建）
投放包 sim 的 tb/filelist 与仓库当前版逐字节一致
```

6.6 repack 重构验证（tb_repack，第2批(1)；原始日志 05_isp_interface/tb_repack_run.log）

```
# CHECK OK   [P1 no-bubble]: wordstream identical (24 words)
# CHECK OK   [P1 no-bubble]: tlast at row-end words {5,11,17,23}
# CHECK OK   [P2 bubble@phase1]: wordstream identical (24 words)
# CHECK OK   [P2 bubble@phase1]: tlast at row-end words {5,11,17,23}
# CHECK OK   [P2 bubble@phase2]: wordstream identical (24 words)
# CHECK OK   [P2 bubble@phase2]: tlast at row-end words {5,11,17,23}
# CHECK OK   [P2 bubble@phase3]: wordstream identical (24 words)
# CHECK OK   [P2 bubble@phase3]: tlast at row-end words {5,11,17,23}
# CHECK OK   [P2 bubble@phase0]: wordstream identical (24 words)
# CHECK OK   [P2 bubble@phase0]: tlast at row-end words {5,11,17,23}
# CHECK OK   [P4 back-to-back frame2]: wordstream identical (24 words)
# CHECK OK   [P4 back-to-back frame2]: tlast at row-end words {5,11,17,23}
# TB_RESULT: PASS  (0 fail)  repack hold-phase + tlast rebuild verified
```

七、已知问题与开放问题（请评审重点）

7.1 外部评审 7 条的处置状态

- **#1 AWB** 按帧更新 — 属实 → 已改（5.2）；第**2**批补强：采样数上下界 + 近黑剔除 + frame_bad 门，边界 TB 38/38；
- **#2** 异常帧丢弃（error 线硬接 0）— 属实 → 第**2**批已接：`S_mipi_last_*` 按帧锁存 → video_in 帧首回退 + AWB 拒采信；
- **#3 FIFO** 满/空标志未用 — 属实（全悬空 / 接了未用）→ 第 3 批（先计数观测）；
- **#4 96→128** 重排不支持间隙 — 属实 → 第**2**批已改：气泡保相位 + repack 内 tlast 重建；新增 post-repack 记分板 TB（PASS）；
- **#5 tlast/ready** 语义 — 属实 → 第1批消固定错位，第**2**批在 **repack** 内重建（与 O_tdata/O_tvalid 同拍）；完整握手重构暂缓；
- **#6** 参数分散 **/ gamma** 未例化 — 属实 → 第 3 批（随视觉链集成）；
- **#7** 饱和度 **v** 位宽溢出（**K≥33**） — 确凿 bug → 已修复（12bit，K=50 PASS）。

7.2 设计取舍（请评审这些参数选择）

取舍	当前值	理由	疑问
AWB 滞后	2 帧（≈33ms@60fps）	上一帧统计→本帧恒定，消除帧内漂移	对运动场景是否可感知？
增益步进	±0.125x/帧	场景切换 ~8 帧收敛，抑制色彩泵动	步进与滞后的组合是否过慢/过快？
增益限幅	0.5x~3.0x	防除法病态（暗场景小分母）与极端修正	上限 3.0x 是否足够覆盖极端色温？
统计抽样	工程版 1/4 像素（原厂设计）+ 剔除近白（rgb_sum<720）与近黑（>45，第2批）	省资源 + 防暗噪声驱动比值病态	1/4 抽样在纹理丰富场景的偏差风险？（相位轮换列为候选）
采信门槛	20480 < 非白拍/帧 < 300000（第2批加下/上界）	防短/空帧与漏 SOF 并帧（720p 并帧 460800 必拒）	门限取舍是否合理？
饱和度	K=1.75x（Q4.4，上限 3.94x）	用户上板定版	与 AWB 的相互作用（见 Q3）

7.3 风险与未覆盖项

- **R1**：DDR/显示路径（video_in/video_out/hdmi_tx）不在任何仿真内——第2批的坏帧门接线只能上板验证；
- **R2**：仿真树的 `isp_top` 为"抽头变体"（与工程版在端口层分叉），整链仿真不覆盖 frame_bad/repack-tlast 接线；
- **R3**：`axis_saturation_rgb96` 存在"独立文件 + isp_top 内嵌"双副本（现靠哈希一致性核验；计划最终并入 `.prj` 源列表）；

- **R4**：交付包内仿真 filelist 使用仓库绝对路径（离开本机无法独立运行）；
- **R5**：厂商 `.al` 与本机 TD GUI 不兼容（PRJ-8641），重建依赖无头流程（已验证可复现）。

7.4 请评审回答的问题清单

- **Q1**：AWB 帧同步化（5.2）的逻辑与边界是否正确？特别检查：上电序列（prev 初值 1 → 商恰为 1.0）、

`stat_beats` 门槛、异常/碎帧时"保持旧增益"的行为、tuser 与统计清零同拍的非阻塞读旧值。

- **Q2**：增益应用段（Q16 乘法 + `[27:24]` 判 255 饱和）与帧同步化增益的配合是否有新的溢出/精度问题？
- **Q3**：无 CCM 的链路里，饱和度 1.75x 是否会放大 AWB 残余色偏？两者先后级（AWB→饱和度）是否合理？
- **Q4**：饱和度位宽修复（5.1，v=12bit）复核：K≤63 时最坏 |v|≈1004/−749，12bit 是否充分？

`t`（16bit）与乘法链还有无其他溢出点？

- **Q5**：`O_tlast = sat_O_tlast` 的对齐程度（repack 未参与 tlast 重排）：将来在 ISP 输出后挂视觉检测模块时，这个边界的正确处理建议？
- **Q6**：#2 异常帧丢弃：顶层 `S_mipi_length_error/lane_error/last_*` 到 `video_in.I_mipi_rx_error` 的"按帧锁存"实现建议（时序关联与复位点）？
- **Q7**：#3 FIFO 观测：溢出/欠载计数 + 欠载补色 + 溢出后等帧重同步的最小实现方案？
- **Q8**：#4 重排模块的间隙兼容改法（保相位 vs 帧边界复位）与无仿真覆盖下的验证方法？

八、复现速查

```
rem ---- 整链仿真（K 可在 TB 的 SAT_K_Q4 改）----
cd D:\fpga-ptz-tracker\08_board_bringup\01_video_passthrough\sim
run_sim.bat
python ..\..\..\tools\dump2png.py . passthrough_frame 0 1 --out sim_out
python ..\..\..\tools\sat_tool.py check --before baseline_oct03\passthrough_frame_0.hex --after passthrou
gh_frame_0.hex --k 28

rem ---- AWB 单元 TB ----
cd D:\fpga-ptz-tracker\08_board_bringup\04_awb_framesync
run_tb.bat

rem ---- 无头重建（改 RTL 后）----
cd D:\fpga-ptz-tracker\08_board_bringup\03_td_rebuild_sat\lab_ex1_build\td_project\camera_to_dsi_display_
Runs\syn_1
D:\TD6.2.1\bin\td_commands_prompt.exe D:\TD6.2.1\doc\scripts\DefaultFlow.tcl
cd ..\phy_1
D:\TD6.2.1\bin\td_commands_prompt.exe D:\TD6.2.1\doc\scripts\DefaultFlow.tcl

rem ---- 烧录（SRAM；hwserver 手动启动必须带 -r）----
D:\SC520_display_burnpack\01_bitstream_flash_direct\burn_camera_sat_sram.bat
```

九、给评审者的提示词（v1.1 复核版，可直接复制发给 AI）

```
你是资深 FPGA 图像通路评审专家。这是同一交付包（安路 PH1P35/HX1P35A，SC520CS 摄像头→HDMI 直通）
评审卷宗的 **v1.1 修订版**——按你上一轮评审意见逐条落实后的版本，请做**复核 + 增量评审**。

上一轮要点与处置（详见文首"v1.1 变更说明"与第七章 7.1 更新）：
- #1 AWB：采样数上下界（20480<beats<300000）+ 近黑剔除（rgb_sum>45）+ frame_bad 拒采信门；
  边界 TB 38/38（含 20480 拒/20481 采、亮白/近黑剔除、超长帧拒、坏帧门、帧中复位）；
- #2 异常帧：S_mipi_last_* 按帧锁存 → video_in.I_mipi_rx_error（帧首回退）+ AWB 门（按你的 Q6 方案）；
- #4/#5/#8 repack：气泡保相位 + tlast 在 repack 内重建（5.9），新增 post-repack byte-stream
  记分板（5.10/6.6：4 相位气泡注入 + tlast 落位 + back-to-back，PASS）；
- #7：v 位宽 10→12bit（K=50 位精确对拍 PASS）；
- 卷宗两处过强表述已更正（"防异常帧污染"、"tlast 已对齐"），风险清单 R1 重写。
- 上板状态：awbfs_b2 已重建待烧（下载器未连接，6.4 已如实标注）；sat_k28 版曾上板验证。

请按以下优先级输出：
1) 复核：对上述每项给"正确 / 仍有问题 / 表述仍不准确"，逐条附依据（模块/信号/代码行为）；
2) 重点检查两处新代码：repack 重构（5.9）的相位/边界；AWB 边界相位（5.4 与 6.1 证据）的实现细节；
3) 对仍未处理项（#3 FIFO 计数与重同步、#6 参数集中管理、1/4 抽样相位轮换）给最小实现建议；
4) 若发现新问题，按 严重度/位置/建议修复 排列。
不要泛泛而谈；每个结论给可核验依据；不同意处直接指出并说明理由。
```